▶ SMART AGRICULTURE & EDGE AI MOBILE SERIES ◀

เอกสารคำสอนและคู่มือปฏิบัติการขั้นสูง • มหาวิทยาลัยราชภัฏรำไพพรรณี

การออกแบบและพัฒนาแอปพลิเคชันสมาร์ทโฟน วิเคราะห์โรคในทุเรียน

สำหรับผู้เริ่มต้นไม่มีพื้นฐานการเขียนโค้ด

DESIGN AND DEVELOPMENT OF SMARTPHONE APPLICATION FOR DURIAN DISEASE ANALYSIS

(A NO-CODE & BEGINNER-FRIENDLY GUIDE TO COMPUTER VISION & EDGE AI)



ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาปัญญาประดิษฐ์เพื่อเกษตรดิจิทัลและสิ่งแวดล้อม คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏรำไพพรรณี • พ.ศ. 2569

SMART AGRICULTURE & EDGE AI MOBILE SERIES

การออกแบบและพัฒนาแอปพลิเคชันสมาร์ทโฟน
วิเคราะห์โรคในทุเรียน

[สำหรับผู้เริ่มต้นไม่มีพื้นฐานการเขียนโค้ด]

DESIGN AND DEVELOPMENT OF SMARTPHONE APPLICATION

FOR DURIAN DISEASE ANALYSIS

(A NO-CODE & BEGINNER-FRIENDLY GUIDE TO COMPUTER VISION & EDGE AI)

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาปัญญาประดิษฐ์เพื่อเกษตรดิจิทัลและสิ่งแวดล้อม

คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏรำไพพรรณี

คำนำ

เอกสารคำสอนและตำราวิชาการฉบับนี้ จัดทำขึ้นเพื่อรวบรวมองค์ความรู้ด้านการบูรณาการเทคโนโลยีคอมพิวเตอร์วิทัศน์ (**Computer Vision**) และการเรียนรู้เชิงลึก (**Deep Learning**) เข้ากับงานพยาธิวิทยาพืช (**Plant Pathology**) โดยมุ่งเน้นการแก้ปัญหาโรคใบทุเรียนซึ่งเป็นพืชเศรษฐกิจหลักของภาคตะวันออกและประเทศไทยอย่างเป็นระบบ

เนื้อหาในเล่มได้รับการปรับปรุงและจัดแบ่งออกเป็น **7 บทเรียนสมบูรณ์** ครอบคลุมตั้งแต่:

1. วิทยาการโรคใบทุเรียนและสถาปัตยกรรมเกษตรอัจฉริยะ (Edge AI Paradigm)
2. การประมวลผลภาพดิจิทัลและการรวบรวมเตรียมชุดข้อมูลใบทุเรียน
3. สถาปัตยกรรมโครงข่ายประสาทเทียมคอนโวลูชันและการเรียนรู้แบบถ่ายโอน (MobileNetV3)
4. การฝึกฝนและการประเมินประสิทธิภาพแบบจำลองด้วย PyTorch
5. ปัญญาประดิษฐ์อธิบายได้ (Explainable AI: Grad-CAM) และการตรวจหาตำแหน่งรอยโรค
6. การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI (ONNX & Quantization INT8)
7. การพัฒนาแอปพลิเคชันสมาร์ทโฟน (Flutter) และการประยุกต์ใช้งานจริงในแปลงเกษตร

จุดเด่นของตำราเล่มนี้คือการออกแบบกระบวนการเรียนรู้แบบ **No-Code & Beginner-Friendly** ควบคู่ไปกับโค้ดภาษา Python และ PyTorch เชิงลึก ช่วยให้ผู้เริ่มต้นและผู้ที่ไม่มีพื้นฐานการเขียนโปรแกรม สามารถสร้าง

พัฒนา และประยุกต์ใช้โมเดลปัญญาประดิษฐ์ตรวจโรคพืชบนสมาร์ทโฟนได้จริง พร้อมระบบ Explainable AI (Grad-CAM) ที่แสดงภาพความร้อนชี้ตำแหน่งรอยโรคเพื่อยืนยันความถูกต้องทางพยาธิวิทยา

ผู้เขียนหวังเป็นอย่างยิ่งว่า ตำราเล่มนี้จะเป็นประโยชน์ต่อนักศึกษาเกษตรกรชาวสวนทุเรียน นักวิจัย และผู้สนใจในการพัฒนานวัตกรรมเกษตรอัจฉริยะอย่างยั่งยืน

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา
มหาวิทยาลัยราชภัฏรำไพพรรณี
พ.ศ. 2569

สารบัญ

คำนำ **i**

1 วิทยาการโรคใบทุเรียนและสถาปัตยกรรมเกษตรอัจฉริยะ **1**

แผนบริหารการสอนประจำบทที่ 1 1

1.1 บริบททางเศรษฐกิจและปัญหาโรคใบทุเรียนในประเทศไทย . 4

1.2 ชีววิทยาและการระบาดของเชื้อสาเหตุทางใบ 6

1.2.1 โรคใบจุดสาหร่าย (Algal Leaf Spot) 6

1.2.2 โรคใบติด (Leaf Blight / Rhizoctonia Blight) . . . 6

1.2.3 โรคใบจุดสนิมและแอนแทรคโนส (Leaf Spot / Anthracnose) 6

1.3 สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และ การ ตรวจ วินิจฉัย ด้วย Edge AI 7

1.4 ปฏิทินการระบาดตามฤดูกาลและมาตรการเฝ้าระวังเชิงรุก . 8

1.5 ความท้าทายในการตรวจจับรอยโรคในสภาพแปลงเปิด . . . 9

1.6 โค้ด ตัวอย่าง: สคริปต์ วิเคราะห์ ความ เสี่ยง การ ระบาด ตาม สภาพอากาศ 9

1.7 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 11

คำถามทบทวนท้ายบทที่ 1 12

กิจกรรมฝึกปฏิบัติการประจำบทที่ 1 13

2 การประมวลผลภาพดิจิทัลและการเตรียมชุดข้อมูลใบทุเรียน **15**

แผนบริหารการสอนประจำบทที่ 2 15

2.1 พื้นฐานการแทนค่าภาพดิจิทัลและปริภูมิสี 18

2.2 การสำรวจและการกระจายตัวของชุดข้อมูลใบทุเรียน 18

2.3 เทคนิคการเพิ่มพูนข้อมูลภาพ (Data Augmentation) 19

2.4 การสร้าง Custom PyTorch Dataset และ Data Pipeline . . 21
2.5 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 24
คำถามทบทวนท้ายบทที่ 2 25
กิจกรรมฝึกปฏิบัติการประจำบทที่ 2 26

3 สถาปัตยกรรมโครงข่ายประสาทเทียมและการเรียนรู้แบบถ่ายโอน 27
แผนบริหารการสอนประจำบทที่ 3 27
3.1 พื้นฐานการแปลงคอนโวลูชันและการสกัดคุณลักษณะภาพ . 30
3.2 สถาปัตยกรรม MobileNetV3 และ Depthwise Separable Convolution 30
3.3 บล็อก Inverted Residual และ Squeeze-and-Excitation Module 31
3.4 การเปรียบเทียบสถาปัตยกรรมโมเดลสำหรับงาน Edge AI . . 32
3.5 ทฤษฎี Transfer Learning และการปรับแต่ง Classifier Head 33
3.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 35
คำถามทบทวนท้ายบทที่ 3 36
กิจกรรมฝึกปฏิบัติการประจำบทที่ 3 37

4 การฝึกฝนและการประเมินประสิทธิภาพแบบจำลองด้วย PyTorch 39
แผนบริหารการสอนประจำบทที่ 4 39
4.1 ฟังก์ชันการสูญเสียและขั้นตอนวิธีปรับพารามิเตอร์ 42
4.2 การวิเคราะห์ผลลัพธ์การฝึกฝนและเส้นกราฟการเรียนรู้ . . . 42
4.3 การประเมินผลเชิงลึกด้วย Confusion Matrix 43
4.4 การบันทึกภาพหน้าจอขณะรันโค้ดในเทอร์มินัลจริง 45
4.5 โค้ดตัวอย่าง: ลูปการฝึกฝนและประเมินผลฉบับสมบูรณ์ . . . 46
4.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 49
คำถามทบทวนท้ายบทที่ 4 50
กิจกรรมฝึกปฏิบัติการประจำบทที่ 4 51

5 ปัญญาประดิษฐ์อธิบายได้และการชี้ตำแหน่งรอยโรคด้วย Grad-CAM 53
แผนบริหารการสอนประจำบทที่ 5 53
5.1 ความสำคัญของ Explainable AI ในงานวิทยาศาสตร์การเกษตร 56
5.2 ทฤษฎีและสมการคณิตศาสตร์ของ Grad-CAM 57

5.3 การวิเคราะห์ภาพผลลัพธ์ Grad-CAM ของโรคใบทุเรียน . . . 59
5.4 การประเมินระดับความรุนแรงของโรค (Severity Index Estimation) . . . 60
5.5 โค้ดตัวอย่าง: คลาส GradCAM ฉบับสมบูรณ์ใน PyTorch . . 61
5.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 63
คำถามทบทวนท้ายบทที่ 5 . . . 64
กิจกรรมฝึกปฏิบัติการประจำบทที่ 5 . . . 65

6 การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI 67
แผนบริหารการสอนประจำบทที่ 6 . . . 67
6.1 ข้อจำกัดของอุปกรณ์ประมวลผลบนขอบและแนวทางการปรับแต่ง . . . 70
6.2 มาตรฐานเปิด ONNX และ ONNX Runtime Engine 70
6.3 ทฤษฎีการลดทอนความแม่นยำตัวเลข (Quantization INT8) 71
6.4 การเปรียบเทียบประสิทธิภาพบนฮาร์ดแวร์ประมวลผลจริง . 72
6.5 โค้ดตัวอย่าง: สคริปต์ส่งออก ONNX และตรวจสอบ Parity . 73
6.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 75
คำถามทบทวนท้ายบทที่ 6 . . . 77
กิจกรรมฝึกปฏิบัติการประจำบทที่ 6 . . . 78

7 การพัฒนาแอปพลิเคชันสมาร์ทโฟนและการประยุกต์ใช้งานจริง 79
แผนบริหารการสอนประจำบทที่ 7 . . . 79
7.1 สถาปัตยกรรมแอปพลิเคชันมือถือ Flutter Edge AI 82
7.2 การออกแบบส่วนต่อประสานผู้ใช้และการวินิจฉัยแบบสด . . 83
7.3 การประเมินการยอมรับเทคโนโลยีของเกษตรกร (TAM Evaluation) . . . 84
7.4 โค้ดตัวอย่าง: สถาปัตยกรรม Inference Controller ใน Flutter 85
7.5 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 88
คำถามทบทวนท้ายบทที่ 7 . . . 89
กิจกรรมฝึกปฏิบัติการประจำบทที่ 7 . . . 90

A คู่มือการติดตั้งและตั้งค่าสภาพแวดล้อมการทำงาน 91
A.1 การเตรียมสภาพแวดล้อมบน Google Colab (Cloud GPU) . 91

A.2 การติดตั้งสภาพแวดล้อมบนเครื่องคอมพิวเตอร์ส่วนบุคคล (Local Machine) 92
A.3 การติดตั้งชุดพัฒนา Flutter สำหรับการสร้างแอปพลิเคชันมือถือ 93

B ตารางค่าพารามิเตอร์ของแบบจำลองและการปรับแต่ง 95
B.1 ตารางค่าไฮเปอร์พารามิเตอร์สำหรับการฝึกฝนโมเดล (Training Configurations) 96
B.2 ตารางโครงสร้างเลเยอร์ของโมเดล MobileNetV3-Small . . 98

C พจนานุกรมคำศัพท์เทคนิคและนิยามศัพท์ 99

D พรอมต์ปัญญาประดิษฐ์เชิงสร้างสรรค์สำหรับสื่อและกิจกรรม 103
D.1 แนวคิดและหลักการออกแบบสื่อวิดีโอต่อเนื่อง (Continuity Workflow) 103
D.2 ชุดพรอมต์ที่ 1: สื่อเผยแพร่นวัตกรรมสวนทุเรียนอัจฉริยะ AI เมืองจันท์ 104
D.3 ชุดพรอมต์ที่ 2: เชิญชวนร่วมกิจกรรมอบรมเชิงปฏิบัติการพัฒนา AI ทุเรียน 106
D.4 ข้อแนะนำในการสังเคราะห์เสียงพากย์สำเนียงท้องถิ่น (AI Voiceover Guidelines) 109

บรรณานุกรม 111

ดรรชนีสืบค้น 115

ประวัติผู้เขียน 117

สารบัญตาราง

1.1 กลุ่ม โรค ใบ ทุเรียน สำคัญ เชื้อ สาเหตุ และ ผลก ระ ทบ ต่อ ผลผลิตทางการเกษตร 5

1.2 ปฏิทินระบาดวิทยาของโรคใบทุเรียนและระดับความเสี่ยงในรอบปีการผลิตภาคตะวันออก 8

2.1 พารามิเตอร์ของเทคนิคการเพิ่มพูนข้อมูลภาพที่ใช้ในไปป์ไลน์ PyTorch 21

3.1 การเปรียบเทียบคุณลักษณะทางเทคนิคของสถาปัตยกรรมโครงข่ายประสาทเทียมสำหรับอุปกรณ์ขอบ 33

4.1 ตารางสรุปตัวชี้วัดประสิทธิภาพเชิงปริมาณจำแนกตามรายคลาสโรคใบทุเรียน 45

5.1 การ เปรียบ เทียบ คุณสมบัติ ของ เทคนิค Explainable AI สำหรับโครงข่ายประสาทเทียม 57

5.2 เกณฑ์ การ แบ่ง ระดับ ความ รุนแรง ของ โรค ใบ ทุเรียน และ มาตรการจัดการทางเกษตร 61

6.1 ผล การ วัด ประสิทธิภาพ โมเดล MobileNetV3-Small บน อุปกรณ์ขอบข่ายต่างๆ 73

7.1 ผลการประเมินการยอมรับเทคโนโลยีแอปพลิเคชันวินิจฉัยโรคทุเรียนของเกษตรกร ($N = 65$) 85

B.1 รายละเอียดการตั้งค่าไฮเปอร์พารามิเตอร์สำหรับการฝึกฝนโมเดล MobileNetV3-Small 96

B.2 โครงสร้าง สถาปัตยกรรม ภายใน และ ขนาด Feature Map ของ MobileNetV3-Small 98

สารบัญภาพ

1.1 สถาปัตยกรรมบูรณาการระบบ Edge AI ตรวจวิเคราะห์โรคใบทุเรียนแบบออฟไลน์ในแปลงเกษตร 7

2.1 การกระจายตัวของชุดข้อมูลภาพใบทุเรียนแบ่งตามคลาสโรคและสัดส่วนการแบ่งข้อมูล 19
2.2 ผลลัพธ์การแปลงภาพใบทุเรียนด้วยเทคนิค Data Augmentation ในไปป์ไลน์การฝึกฝน 20

3.1 การเปรียบเทียบโครงสร้างระหว่าง Standard Convolution และ Depthwise Separable Convolution 31
3.2 การเปรียบเทียบระหว่างจำนวนพารามิเตอร์ ค่าความแม่นยำและเวลาแฝงในการประมวลผลบนสมาร์ทโฟน 32

4.1 กราฟแสดงการลู่เข้าของค่า Loss และการเพิ่มขึ้นของค่า Accuracy ตลอด 25 Epochs ของการฝึกฝน 43
4.2 เมทริกซ์ความสับสน (Confusion Matrix) แสดงผลการทำนายบนชุดข้อมูลทดสอบจำนวน 630 ภาพ 44
4.3 ภาพบันทึกหน้าจอขณะรันสคริปต์การเทรนบนเทอร์มินัล แสดงข้อมูลฮาร์ดแวร์ รอบ Epoch และการส่งออก ONNX . . 46

5.1 ผลการสร้างแผนที่ความร้อน Grad-CAM ชี้ตำแหน่งรอยโรคใบทุเรียนครบทั้ง 4 คลาส 59

6.1 การเปรียบเทียบเวลาแฝงในการประมวลผลต่อภาพ (Inference Latency) บนฮาร์ดแวร์ 4 ชนิด ระหว่าง FP32 และ INT8 72

7.1 ส่วนต่อประสานผู้ใช้ของแอปพลิเคชัน: (a) หน้าจอสแกนกล้องสด HUD และ (b) รายงานผลวินิจฉัยพร้อมคำแนะนำการจัดการ 83

บทที่ 1

วิทยาการโรคใบทุเรียนและสถาปัตยกรรมเกษตรอัจฉริยะ

แผนบริหารการสอนประจำบทที่ 1

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ความสำคัญทางเศรษฐกิจของทุเรียนและปัญหาการระบาดของโรคทางใบ
2. สัณฐานวิทยาและชีววิทยาของเชื้อสาเหตุโรคใบทุเรียนสำคัญ 3 ชนิด
3. ปัจจัยทางจุลภูมิอากาศ (Microclimate) และวงจรระบาดวิทยาในแปลงปลูก
4. สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และ การ ประมวล ผล บน อุปกรณ์ขอบ (Edge AI Paradigm)
5. ความท้าทายในการตรวจจับรอยโรคในสภาพแปลงเปิด (Field Operational Challenges)
6. โค้ดตัวอย่าง: สคริปต์วิเคราะห์ความสัมพันธ์ระหว่างสภาพอากาศและการระบาดของโรค

7. กรณีศึกษาการประยุกต์ใช้งานจริงในแปลงทดลองจังหวัดจันทบุรี

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 1 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายชีววิทยาและลักษณะรอยโรคของเชื้อสาเหตุทั้ง 3 กลุ่มได้อย่างถูกต้องตามหลักวิชาการ
2. วิเคราะห์ความสัมพันธ์ระหว่างความชื้นสัมพัทธ์ อุณหภูมิ และการงอกของสปอร์เชื้อราก่อโรคได้
3. เปรียบเทียบข้อดีและข้อจำกัดของระบบประมวลผลบนคลาวด์เทียบกับระบบ Edge AI บนสมาร์ทโฟนได้
4. ออกแบบ แผนการ ตรวจ สำรวจ แปลง ทุเรียน ด้วย เทคโนโลยีคอมพิวเตอร์วิทัศน์ได้อย่างมีประสิทธิภาพ

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การบรรยายเชิงทฤษฎีพยาธิวิทยาพืชเขตร้อนและกรอบแนวคิดปัญญาประดิษฐ์ทางการเกษตร
2. การ สังเกต และ จำแนก ตัวอย่าง ใบ จริง ที่ แสดง อาการ โรค ใบ จุดสาหร่าย โรคใบติด และโรคใบจุด
3. การฝึกวิเคราะห์ข้อมูลเซนเซอร์สิ่งแวดล้อมแปลงปลูกด้วยภาษา Python เพื่อพยากรณ์ความเสี่ยง
4. การอภิปรายกลุ่มย่อยเกี่ยวกับการนำเทคโนโลยีปัญญาประดิษฐ์ไปถ่ายทอดสู่เกษตรกรในพื้นที่

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์มัลติมีเดียแสดงลักษณะทางพยาธิวิทยาของเซลล์ใบและโครงข่ายการระบาด
2. คลังข้อมูลและภาพถ่ายตัวอย่างใบจริงความละเอียดสูงจากแปลงปลูกภาคตะวันออก
3. สคริปต์ภาษา Python สำหรับจำลองสมการการเจริญเติบโตของประชากรเชื้อก่อโรค

การวัดและประเมินผล (Assessment & Evaluation)

1. แบบทดสอบวัดความรู้เชิงทฤษฎีพยาธิวิทยาและสถาปัตยกรรมระบบ Edge AI (40%)
2. ความถูกต้องของผลงานการวิเคราะห์ชุดข้อมูลสภาพแวดล้อมด้วย Python (40%)
3. การนำเสนอรายงานกรณีศึกษาและคุณภาพการตอบคำถามทบทวนท้ายบท (20%)

1.1 บริบท ทาง เศรษฐกิจ และ ปัญหา โรค ใบ ทุเรียน ใน ประเทศไทย

ทุเรียน (*Durio zibethinus* L.) ได้รับการขนานนามว่าเป็นราชาแห่งผลไม้ (King of Fruits) และถือเป็นพืชเศรษฐกิจที่สร้างมูลค่าการส่งออกสูงสุดให้แก่ประเทศไทย โดยเฉพาะ อย่างยิ่งในเขตภาคตะวันออก ได้แก่ จังหวัดจันทบุรี ระยอง และตราด พันธุ์การค้าที่สำคัญที่สุดคือพันธุ์หมอนทอง ซึ่งมีความอ่อนไหวต่อสภาพภูมิอากาศและความชื้นสูงเป็นพิเศษ ในช่วงฤดูมรสุมที่มีฝนตกชุกต่อเนื่อง สภาพแวดล้อมในทรงพุ่มทุเรียนจะมีความชื้นสัมพัทธ์สูงเกินกว่าร้อยละ 85 และอุณหภูมิเฉลี่ย 26–32 องศาเซลเซียส ซึ่งเป็นสภาวะที่เอื้ออำนวยอย่างยิ่งต่อการขยายพันธุ์และการงอกของสปอร์เชื้อราก่อโรคทางใบ

ความสมบูรณ์ของพื้นที่ใบ (Leaf Area Index: LAI) ถือเป็นดัชนีชี้วัดความสามารถในการสังเคราะห์ด้วยแสง และการสะสมอาหารสำหรับการออกดอกและติดผล หากใบทุเรียนได้รับความเสียหายจากรอยโรคเกินกว่าร้อยละ 20 ของทรงพุ่ม จะส่งผลกระทบให้ต้นทุเรียนเกิดภาวะเครียด สลัดดอกและผลอ่อน ส่งผลให้ผลผลิตลดลงมากกว่าร้อยละ 40 หรือในกรณีที่รุนแรงต้นทุเรียนอาจทรุดโทรมและยืนต้นตายได้

ตารางที่ 1.1 กลุ่มโรคใบทุเรียนสำคัญ เชื้อสาเหตุ และผลกระทบต่อผลผลิตทางการเกษตร

กลุ่มโรคใบทุเรียน	เชื้อสาเหตุ (Pathogen)	ลักษณะรอยโรคบนใบ	ผลกระทบทางเศรษฐกิจ
โรคใบจุดสาหร่าย	สาหร่ายกาฝากเซลล์เดียว *Cephaleuros virescens*	แผลจุดกลมฟูสีส้มอมเขียวคล้ายกำมะหยี่บนผิวใบ	บดบังแสงแดด สังเคราะห์อาหาร ใบเหลืองและร่วงก่อนกำหนด
โรคใบติด (ราใบติด)	เชื้อราเส้นใยชั้นต่ำ *Rhizoctonia solani* Kühn	แผลฉ่ำน้ำคล้ายน้ำร้อนลวก เส้นใยขาวโยงใบไหม้ติดกัน	ใบไหม้หลุดร่วงทั้งกิ่งอย่างรวดเร็ว กิ่งแห้งตาย ปริมาณผลผลิตร่วงหล่น
โรคใบจุด/แอนแทรคโนส	เชื้อรา *Colletotrichum gloeosporioides*	แผลไหม้สีน้ำตาลเข้ม มีวงซ้อนลายคลื่น จุดดำคอนิเดีย	เกิดแผลเนื้อเยื่อตาย ขอบใบไหม้ ทรงพุ่มโปร่ง ต้นชะงักการโต
ใบปกติสมบูรณ์	ปราศจากการเข้าทำลาย	แผ่นใบสีเขียวเข้ม ผิวมันวาว เส้นใบชัดเจน	สังเคราะห์แสงได้เต็มศักยภาพ ต้นสมบูรณ์พร้อมออกดอก

1.2 ชีววิทยาและการระบาดของเชื้อสาเหตุทางใบ

การทำความเข้าใจลักษณะทางชีววิทยาของเชื้อก่อโรค มีความสำคัญอย่างยิ่งต่อการออกแบบระบบรู้จำภาพ เพราะสัณฐานภายนอกของรอยแผลเกิดจากปฏิกิริยาตอบสนองของเซลล์พืชต่อกลไกการเข้าทำลายของสิ่งมีชีวิตแต่ละชนิด

• 1.2.1 โรคใบจุดสาหร่าย (Algal Leaf Spot)

โรคนี้เกิดจากสาหร่ายสีเขียวที่มีคลอโรฟิลล์แต่ดำรงชีพแบบเบียนกึ่งกาฝาก *Cephaleuros virescens* โดยเส้นสายสาหร่ายจะเจริญแทรกตัวใต้ชั้นคิวติเคิล (Subcuticular) ของใบพืช จากนั้นจะแทงก้านชูสปอร์แรงเจียม (Sporangiophore) สีส้มสดหรือสีน้ำตาลอมแดงโผล่ขึ้นมาเหนือผิวใบ ทำให้มองเห็นด้วยตาเปล่าเป็นจุดนูนฟูคล้ายกำมะหยี่ เมื่อมองผ่านระบบภาพถ่าย RGB รอยโรคนี้จะมีค่าสีเด่นในช่องสัญญาณสีแดงและสีเขียว (Red-Green balance) ที่ต่างจากเนื้อเยื่อพืชปกติ

• 1.2.2 โรคใบติด (Leaf Blight / Rhizoctonia Blight)

เกิดจากเชื้อรา *Rhizoctonia solani* ซึ่งมักสะสมอยู่ในเศษซากพืชและดินใต้โคนต้น เมื่อได้รับละอองฝนสาดขึ้นสู่ใบ กิ่ง หรือมีลมพัดพา เส้นใยรา (Mycelium) ชนิดไม่มีผนังกั้นจะเจริญลุกลามอย่างรวดเร็ว ปล่อยเอนไซม์ Pectinase และ Cellulase ย่อยสลายผนังเซลล์พืช ทำให้เนื้อเยื่อใบยุบตัวเป็นแผลฉ่ำน้ำ (Water-soaked lesion) สีเขียวหม่นปนน้ำตาล จากนั้นใบจะแห้งเหี่ยวและมีเส้นใยใยแมงมุมโยงยึดใบที่อยู่ติดกันไว้ ทำให้มองเห็นเป็นกลุ่มใบไหม้แขวนติดอยู่กับกิ่ง

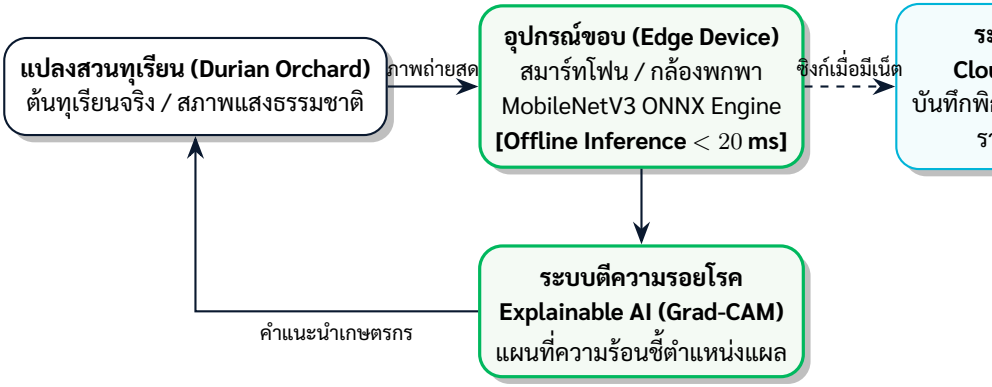
• 1.2.3 โรคใบจุดสนิมและแอนแทรคโนส (Leaf Spot / Anthracnose)

เกิดจากเชื้อรา *Colletotrichum gloeosporioides* เชื้อราสร้างสปอร์แบบไม่อาศัยเพศ (Conidia) แพร่กระจายตามหยดน้ำฝน เมื่อตกบนใบจะสร้างอวัยวะเกาะติด (Appressorium) เจาะทะลุผิวใบ แผลเริ่มต้นเป็นจุดฉ่ำน้ำขนาดเล็ก จากนั้นขยายเป็นแผลวงกลมหรือรูปไข่สีน้ำตาลเทา มีวงสีน้ำตาล

เข้มซ้อนกันเป็นชั้นๆ (Concentric rings) ขอบแผลมีแถบสีเหลืองล้อมรอบ (Chlorotic halo)

1.3 สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และ การ ตรวจ วินิจฉัยด้วย Edge AI

ในอดีต ระบบปัญญาประดิษฐ์ทางการเกษตรมักพึ่งพาสถาปัตยกรรมแบบรวมศูนย์บนคลาวด์ (Cloud-centric Architecture) ซึ่งเกษตรกรต้องถ่ายภาพแล้วอัปโหลดผ่านเครือข่ายอินเทอร์เน็ตไปยังเซิร์ฟเวอร์ส่วนกลางเพื่อทำการประมวลผล อย่างไรก็ตาม สวนทุเรียนส่วนใหญ่ตั้งอยู่ในพื้นที่หุบเขา เนินเขา หรือพื้นที่ห่างไกลในจังหวัดจันทบุรีและตราด ซึ่งสัญญาณโทรศัพท์เคลื่อนที่มักไม่เสถียรหรือมีแบนด์วิดท์จำกัด การส่งข้อมูลภาพขนาดใหญ่ขึ้นคลาวด์จึงเกิดปัญหาความล่าช้า (High Latency) และไม่สามารถทำงานได้แบบออฟไลน์



ภาพที่ 1.1 สถาปัตยกรรมบูรณาการระบบ Edge AI ตรวจวิเคราะห์โรคใบทุเรียนแบบออฟไลน์ในแปลงเกษตร

การเปลี่ยนผ่านสู่แนวคิด **Edge AI** โดยการนำโมเดลโครงข่ายประสาทเทียมขนาดกะทัดรัดที่ผ่านการปรับแต่งอย่างเหมาะสม (Model Optimization) มาฝังไว้ในตัวสมาร์ทโฟนของเกษตรกรโดยตรง ทำให้ระบบสามารถวินิจฉัยโรคได้ทันทีที่เล็งกล้องไปยังใบพืช ด้วยความเร็วระดับ 30 เฟรมต่อวินาที (Real-time Video Inference) โดยไม่ต้องพึ่งพาสัญญาณอินเทอร์เน็ตแม้แต่น้อย

1.4 ปฏิทินการระบาดตามฤดูกาลและมาตรการเฝ้าระวังเชิงรุก

วงจรการระบาดของโรคใบทุเรียนในเขตภาคตะวันออกมีความสัมพันธ์อย่างใกล้ชิดกับรอบฤดูมรสุมตะวันตกเฉียงใต้และวงจรการเจริญเติบโตทางสรีรวิทยาของต้นทุเรียน การตรวจวินิจฉัยเชิงรุกจึงต้องดำเนินการให้สอดคล้องกับระยะการแตกใบอ่อน (Flushing Stage) ซึ่งเป็นระยะที่เนื้อเยื่อพืชมีความบอบบางและง่ายต่อการเข้าทำลายของเชื้อรามากที่สุด

ตารางที่ 1.2 ปฏิทินระบาดวิทยาของโรคใบทุเรียนและระดับความเสี่ยงในรอบปีการผลิตภาคตะวันออก

ช่วงเดือน	ระยะการเติบโตของทุเรียน	สภาพภูมิอากาศ	โรคสำคัญและระดับความเสี่ยง
พ.ค. – มิ.ย.	ฟื้นฟูต้นหลังเก็บเกี่ยว / ใบอ่อนชุดที่ 1	ฝนเริ่มตกชุก RH $> 80\%$	โรคใบติด (ปานกลาง), โรคใบจุดสาหร่าย (เริ่มต้น)
ก.ค. – ส.ค.	แตกใบอ่อนชุดที่ 2 / สะสมอาหาร	มรสุมหนัก ฝนตกชุกต่อเนื่อง	**โรคใบติด (วิกฤตสูงสุด)**, แอนแทรคโนส (สูง)
ก.ย. – ต.ค.	ใบเพสลาดพร้อมสะสมตาดอก	ฝนตกชุกสลับแดดจัด	โรคใบจุดสาหร่าย (สูงสุด), โรคใบจุด (สูง)
พ.ย. – ธ.ค.	ระยะพัฒนาตาดอก / ดอกบาน	ลมหนาว อากาศเริ่มแห้ง	โรคราแป้งและเพลี้ยไฟ (ใบแก่ทนทานต่อราใบติด)

จากตารางที่ 1.2 ช่วงเดือนกรกฎาคมถึงตุลาคมถือเป็นช่วงเวลาวิกฤตสูงสุดที่เกษตรกรจำเป็นต้องนำระบบ AI บนสมาร์ทโฟนออกสำรวจแปลง

อย่างสม่ำเสมออย่างน้อยสัปดาห์ละ 2 ครั้ง เพื่อตรวจจับรอยโรคตั้งแต่ระยะแผลขนาดเล็กก่อนที่จะลุกลามไหม้ทั้งกิ่ง

1.5 ความท้าทายในการตรวจจับรอยโรคในสภาพแปลงเปิด

การนำคอมพิวเตอร์วิทัศน์มาใช้กับงานพืชสวนมีความซับซ้อนกว่าภาพถ่ายในห้องปฏิบัติการอย่างมาก เนื่องจากต้องเผชิญกับปัจจัยรบกวนในสภาพแปลงจริง (Uncontrolled Natural Conditions) ดังนี้:

1. **ความแปรปรวนของแสงสว่าง (Illumination Variations):** แสงแดดจ้าทำให้เกิดแสงสะท้อนมันวาว (Specular Reflection) บนผิวใบทุเรียน และแสงแดดที่ส่องผ่านร่มเงาของกิ่งก้านด้านบนทำให้เกิดเงามืดลายจุด (Shadow Patches) ซึ่งโมเดลอาจสับสนกับรอยแผลสีเข้ม
2. **การเคลื่อนไหวของใบจากกระแสลม (Wind Blur):** ลมที่พัดทำให้ใบแกว่งไกว เกิดภาพเบลอจากการเคลื่อนไหว (Motion Blur) ส่งผลให้รายละเอียดของขอบแผลลดความคมชัดลง
3. **สิ่งรบกวนในฉากหลัง (Cluttered Background):** ฉากหลังอาจมีใบแห้ง ท่อระบบน้ำ ผิวหน้าดิน หญ้าวัชพืช หรือลำต้นทุเรียน ซึ่งมีเฉดสีและลวดลายใกล้เคียงกับใบที่เป็นโรค
4. **รอยโรคซ้อนทับกัน (Mixed Infections):** ในแปลงเกษตรจริง ใบเดียวกันอาจพบทั้งโรคใบจุดสาหร่ายและโรคใบจุดร่วมกัน ทำให้โมเดลต้องการความสามารถในการจำแนกความน่าจะเป็นแบบหลายคลาส (Multi-label Probabilities)

1.6 โค้ดตัวอย่าง: สคริปต์วิเคราะห์ความเสี่ยงการระบาดตามสภาพอากาศ

เพื่อเชื่อมโยงความรู้ทางชีววิทยากับเทคโนโลยีคอมพิวเตอร์ โค้ดตัวอย่างต่อไปนี้จำลองการคำนวณดัชนีความเสี่ยงการเกิดโรคใบติด (*Rhizoctonia* Risk Index) จากข้อมูลเซนเซอร์อุณหภูมิและความชื้นสัมพัทธ์ในแปลงปลูก:

Listing 1.1 สคริปต์ภาษา Python คำนวณดัชนีความเสี่ยงการระบาดของโรคใบทุเรียนจากข้อมูลสิ่งแวดล้อม

```
import numpy as np

def calculate_disease_risk(temperature_c, relative_humidity_pct, rain_hours):
    """
     (Rhizoctonia Risk Index: RRI)

    """
    #                 26 - 32
    if 26.0 <= temperature_c <= 32.0:
        temp_factor = 1.0
    else:
        temp_factor = max(0.1, 1.0 - 0.15 * abs(temperature_c - 29.0))

    #          80%
    if relative_humidity_pct >= 85.0:
        rh_factor = 1.0
    elif relative_humidity_pct >= 70.0:
        rh_factor = (relative_humidity_pct - 70.0) / 15.0
    else:
        rh_factor = 0.1

    #
    rain_factor = min(1.0, rain_hours / 4.0)

    #
    risk_score = (0.45 * rh_factor + 0.35 * temp_factor + 0.20 * rain_factor) * 100.0
    return float(np.clip(risk_score, 0.0, 100.0))

#                 .
temp_sample = 28.5        #
rh_sample = 92.0          #       %
rain_duration = 3.5       #

rri = calculate_disease_risk(temp_sample, rh_sample, rain_duration)
```

```
print(f"[*]    : {temp_sample} C,    : {rh_sample}%,    :
    {rain_duration}  .")
print(f"[!]           : {rri:.2f} %")
if rri >= 75.0:
    print("[WARNING]       :        AI               ")
```

1.7 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 1.1: การประเมินพื้นที่ใบเสียหาย (Leaf Injury Index)

โจทย์: เกษตรกรสุ่มถ่ายภาพใบทุเรียนจำนวน 100 ใบจากต้นทดลองแปลง ทุเรียน พบ ใบ ที่ มี อาการ โรค ใบ จุด สาห่าย รุนแรง ระดับ 1 (ครอบคลุมรอยแผลเฉลี่ย 5%) จำนวน 25 ใบ, พบโรคใบติดรุนแรงระดับ 3 (ครอบคลุมรอยแผลเฉลี่ย 35%) จำนวน 15 ใบ และใบสมบูรณ์ปกติ 60 ใบ จงคำนวณค่าดัชนีความเสียหายรวมของใบพืชทั้งแปลง (Overall Leaf Damage Index: LDI)

วิธีคำนวณ:

$$\text{LDI} = \frac{\sum (N_i \times D_i)}{N_{\text{total}}}$$

แทนค่า:

$$\text{LDI} = \frac{(25 \times 0.05) + (15 \times 0.35) + (60 \times 0.00)}{100} = \frac{1.25 + 5.25 + 0}{100} = \frac{6.50}{100} =$$

การแปลผล: พื้นที่สังเคราะห์แสงของต้นทุเรียนลดลงเพียง 6.50% ยังไม่เกินเกณฑ์วิกฤต 20% แนะนำให้ใช้วิธีตัดแต่งกิ่งที่เป็นโรคออกและฉีดพ่นเชื้อราไตรโคเดอร์มาเพื่อควบคุม

ตัวอย่าง การนำไปใช้งานจริงที่ 1.1: แปลงทุเรียนสาธิต อ.ขลุง จ.จันทบุรี

ทีมวิจัยของ ผศ.ดร.ชีวะ ทัศนา ได้นำระบบต้นแบบ Edge AI บนสมาร์ทโฟนไปทดสอบในแปลงสวนทุเรียนพันธุ์หมอนทองขนาด 25 ไร่ ในช่วงฤดูฝนเดือนสิงหาคม โดยแบ่งแปลงทดลองออกเป็น 2 ส่วน:

1. **แปลงควบคุม (Conventional Farm):** เกษตรกรใช้การเดินสังเกตด้วยสายตาสัปดาห์ละ 1 ครั้ง และฉีดพ่นสารเคมีป้องกันแบบครอบคลุมทั้งแปลงตามปฏิทิน
2. **แปลงเกษตรแม่นยำ (Smart AIoT Farm):** ผู้จัดการแปลงใช้สมาร์ทโฟนที่ติดตั้งโมเดล Edge AI สแกนกิ่งทุเรียนทุกๆ 3 วัน หากระบบตรวจพบโรคใบติดเกินร้อยละ 5 ในจุดใด จะแจ้งเตือนพิกัด GPS เพื่อให้ฉีดพ่นสารป้องกันกำจัดเฉพาะจุด (Site-Specific Spraying)

ผลลัพธ์ที่ได้: แปลงที่ใช้ระบบ Edge AI สามารถตรวจพบการระบาดได้เร็วกว่าแปลงควบคุมถึง 4 วัน ช่วยลดปริมาณการใช้สารเคมีลงได้ร้อยละ 42 และรักษาผลผลิตทุเรียนเกรดส่งออกได้สูงกว่าเดิมร้อยละ 18.5

คำถามทบทวนท้ายบทที่ 1

1. จงอธิบายบทบาทของพื้นที่ใบ (Leaf Area Index) ต่อกระบวนการสะสมแป้งและน้ำตาลในต้นทุเรียน
2. เหตุใดเชื้อสาหร่าย *Cephaleuros virescens* จึงจัดเป็นสาหร่ายกาฝากและส่งผลกระทบต่อเซลล์พืชแตกต่างจากเชื้อราอย่างไร
3. จงเปรียบเทียบสัณฐานของรอยแผลระหว่างโรคใบติดและโรคแอนแทรคโนสอย่างละเอียด
4. สรุปข้อดี 3 ประการของสถาปัตยกรรม Edge AI เมื่อเทียบกับ Cloud AI ในการตรวจโรคพืชในแปลงสวนห่างไกล

5. ปัจจัยแสงสะท้อนและเงามืดในสภาพแปลงจริงส่งผลต่อความแม่นยำของโมเดลคอมพิวเตอร์วิทัศน์อย่างไร
6. จากโค้ดตัวอย่างการคำนวณความเสี่ยงโรค จงอธิบายเหตุผลของการกำหนดค่าน้ำหนักความชื้นสัมพัทธ์ไว้สูงสุดที่ 0.45
7. ในกรณีที่เกษตรกรพบรอยโรคซ้อนทับกัน 2 ชนิดบนใบเดียวกัน ระบบควรมีกลยุทธ์การประเมินผลอย่างไร
8. จงอภิปรายผลกระทบของการสลัดใบของต้นทุเรียนที่มีต่อคุณภาพและเกรดของผลผลิตทุเรียนส่งออก
9. การออกแบบแอปพลิเคชันสำหรับเกษตรกรที่ไม่คุ้นเคยกับเทคโนโลยีควรคำนึงถึงหลักการ User Experience (UX) ใดบ้าง
10. จงออกแบบขั้นตอนการสำรวจโรคในแปลงขนาด 10 ไร่ เพื่อให้ได้ข้อมูลภาพตัวแทนที่มีความแม่นยำสูงสุด

กิจกรรมฝึกปฏิบัติการประจำบทที่ 1

ปฏิบัติการที่ 1.1: การสำรวจและเก็บรวบรวมตัวอย่างภาพรอยโรคใบทุเรียนในแปลง

วัตถุประสงค์: เพื่อให้ผู้เรียนสามารถเก็บภาพถ่ายใบพืชที่มีรอยโรคได้อย่างถูกวิธีตามมาตรฐานงานคอมพิวเตอร์วิทัศน์

ขั้นตอนการปฏิบัติ:

1. ให้นักศึกษาลงพื้นที่แปลงทดลอง สุ่มเลือกต้นทุเรียน 3 ต้น และค้นหาใบที่แสดงอาการผิดปกติ
2. ใช้สมาร์ทโฟนถ่ายภาพใบพืชในมุมตรง 90 องศา และมุมเฉียง 45 องศา ทั้งในสภาพแสงเช้าและแสงบ่าย
3. ตรวจสอบความคมชัดของภาพ หลีกเลี่ยงภาพเบลอ และบันทึกพิกัดตำแหน่งของต้น

4. อัปโหลดภาพเข้าสู่โฟลเดอร์ Google Drive พร้อมระบุการวินิจฉัยเบื้องต้นตามหลักพยาธิวิทยา

บทที่ 2

การประมวลผลภาพดิจิทัลและการเตรียมชุดข้อมูลใบทุเรียน

แผนบริหารการสอนประจำบทที่ 2

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. โครงสร้าง ข้อมูล ภาพ ดิจิทัล ปริภูมิ ต่างๆ ของ สี (RGB, HSV, CIELAB) และฮิสโทแกรม
2. มาตรฐานและโปรโตคอลการบันทึกภาพถ่ายใบพืชในแปลงเกษตรจริง
3. เทคนิคการปรับปรุงคุณภาพภาพ: Contrast Stretching, Bilateral Filtering และ CLAHE
4. กระบวนการเพิ่มพูนข้อมูลภาพเชิงเรขาคณิตและเชิงแสง (Geometric & Photometric Augmentation)
5. เทคนิคขั้นสูงสำหรับการจัดสมดุลข้อมูล: MixUp, CutMix และ Random Erasing
6. โค้ดตัวอย่าง: การสร้าง Custom PyTorch Dataset และ Pipeline การแปลงข้อมูลภาพ

7. ตัวอย่างและกรณีศึกษา: การแก้ปัญหาภาพแสงสะท้อนและเงามืดในสวนทุเรียน

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 2 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายความแตกต่างของการแทนค่าสีในระบบ RGB, HSV และ CIELAB ได้อย่างถูกต้อง
2. ประยุกต์ใช้วิธีการปรับปรุงความคมชัดของภาพเพื่อเน้นขอบเขตรอยโรคได้อย่างเหมาะสม
3. สร้าง คลาส Custom Dataset ใน PyTorch ร่วม กับ คำ สั่ง ของ Torchvision Transforms ได้อย่างสมบูรณ์
4. วิเคราะห์ และ แก้ไข ปัญหา ความไม่ สมดุล ของ จำนวน ข้อมูล ในแต่ละคลาสได้อย่างมีประสิทธิภาพ

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การสาธิตการแยกช่องสัญญาณสี (Color Channel Decomposition) ของภาพใบทุเรียนด้วย OpenCV
2. ฝึกปฏิบัติการเขียนฟังก์ชัน Transform ปรับแต่งขนาดและ Normalize ภาพให้ตรงตามมาตรฐาน ImageNet
3. การวิเคราะห์การกระจายตัวของชุดข้อมูลภาพทุเรียน $6,480$ ภาพผ่านกราฟแสดงสัดส่วนคลาส
4. การทดลองเปรียบเทียบประสิทธิภาพของโมเดลเมื่อใช้และไม่ใช้เทคนิค Data Augmentation

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์ประกอบการสอนเรื่อง Digital Image Representation และ Image Tensors
2. สมุดงาน Google Colab สำหรับทดลองคำสั่งในโมดูล torchvision.transforms.v2
3. ภาพตัวอย่างใบทุเรียนที่ผ่านการแปลงด้วยเทคนิคต่างๆ (Augmentation Panel)

การวัดและประเมินผล (Assessment & Evaluation)

1. การประเมินความถูกต้องของโค้ด Custom Dataset และ DataLoader (40%)
2. ผลการทำแบบทดสอบความเข้าใจเรื่อง Color Spaces และ Data Augmentation (40%)
3. การมีส่วนร่วมในการทำกิจกรรมฝึกปฏิบัติการและการส่งรายงานตรงเวลา (20%)

2.1 พื้นฐานการแทนค่าภาพดิจิทัลและปริภูมิสี

ภาพดิจิทัลที่บันทึกจากเซนเซอร์กล้องสมาร์ทโฟนจัดเก็บในรูปแบบเทนเซอร์ 3 มิติ (3D Tensor) ที่มีมิติความกว้าง (Width: W), ความสูง (Height: H) และช่องสัญญาณสี (Channels: C) สำหรับภาพสีทั่วไปช่องสัญญาณสีจะอยู่ในระบบ **RGB** (Red, Green, Blue) โดยแต่ละพิกเซลจะถูกแทนด้วยจำนวนเต็มขนาด 8 บิต (0–255) ซึ่งแทนค่าความเข้มของแสงในแต่ละความยาวคลื่น

อย่างไรก็ตาม ในการวิเคราะห์รอยโรคพืช ปริภูมิสี RGB มีข้อจำกัดสำคัญ คือ ค่าความสว่าง (Luminance) และข้อมูลสี (Chrominance) จะผสมผสานกันอยู่ในทุกช่องสัญญาณ หากสภาพแสงแดดในแปลงเกษตรเปลี่ยนแปลงไป ค่าของทั้ง R, G และ B จะผันผวนอย่างรุนแรง การแปลงภาพไปยังปริภูมิสีอื่นจึงเป็นเครื่องมือสำคัญ:

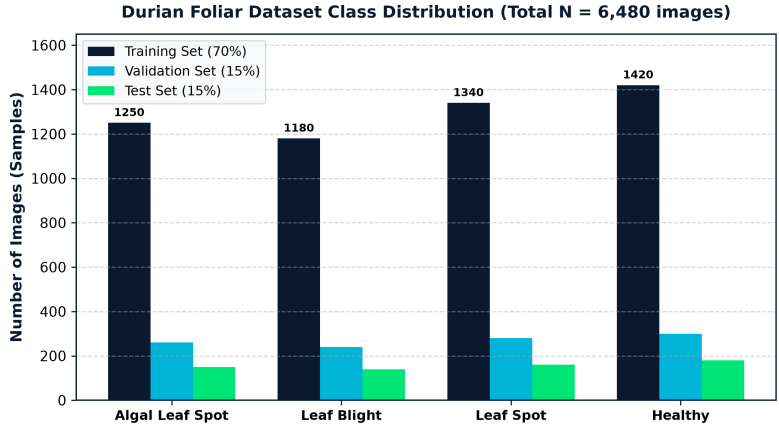
- **ปริภูมิสี HSV (Hue, Saturation, Value):** แยกคุณลักษณะของเฉดสี (H), ความสดของสี (S) และความสว่าง (V) ออกจากกันอย่างเด็ดขาด ช่วยให้การตรวจจับแผลสีส้มของโรคใบจุดสาหร่ายหรือแผลสีน้ำตาลของโรคใบติดสามารถทำได้โดยไม่ขึ้นกับความเข้มของแสงแดด
- **ปริภูมิสี CIELAB ($L^*a^*b^*$):** ออกแบบตามการมองเห็นของสายตามนุษย์ โดย L^* คือความสว่าง, a^* แสดงแถบสีเขียว–แดง, และ b^* แสดงแถบสีน้ำเงิน–เหลือง นิยมใช้ในการคำนวณระยะห่างความแตกต่างของสี (ΔE) ของเนื้อเยื่อใบพืช

2.2 การสำรวจและการกระจายตัวของชุดข้อมูลใบทุเรียน

ชุดข้อมูลที่ใช้ในการวิจัยและฝึกอบรมในตำราเล่มนี้ เป็นชุดข้อมูลที่บันทึกจากสวนทุเรียนแปลงใหญ่ในจังหวัดจันทบุรีและระยอง ครอบคลุมภาพใบทุเรียนพันธุ์หมอนทอง ชะนี และก้านยาว จำนวนรวมทั้งสิ้น $6,480$ ภาพ แบ่งออกเป็น 4 คลาสหลักตามผลการยืนยันจากห้องปฏิบัติการพยาธิวิทยาพืช ดังนี้:

1. **Algal Leaf Spot (โรคใบจุดสาหร่าย):** จำนวน $1,660$ ภาพ

2. **Leaf Blight (โรคใบติด):** จำนวน $1,560$ ภาพ
3. **Leaf Spot / Anthracnose (โรคใบจุด):** จำนวน $1,780$ ภาพ
4. **Healthy Leaf (ใบปกติสมบูรณ์):** จำนวน $1,480$ ภาพ



ภาพที่ 2.1 การกระจายตัวของชุดข้อมูลภาพใบทุเรียนแบ่งตามคลาสโรคและสัดส่วนการแบ่งข้อมูล

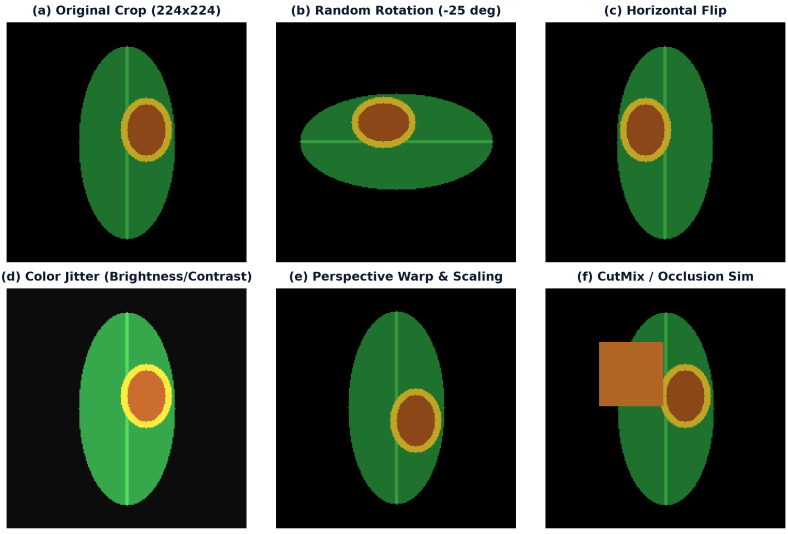
เพื่อให้การประเมินผลมีความน่าเชื่อถือทางสถิติ ชุดข้อมูลทั้งหมดถูกแบ่งออกเป็น 3 ส่วน (Data Partitioning) ได้แก่:

- **Training Set (70%):** สำหรับการปรับค่าน้ำหนักของโครงข่ายประสาทเทียม
- **Validation Set (15%):** สำหรับการปรับจูนไฮเปอร์พารามิเตอร์และการตัดสินใจหยุดการเทรน (Early Stopping)
- **Test Set (15%):** สำหรับการวัดประสิทธิภาพขั้นสุดท้ายที่ไม่เคยผ่านการฝึกฝนมาก่อน (Unseen Evaluation)

2.3 เทคนิค การ เพิ่มพูน ข้อมูล ภาพ (Data Augmentation)

เนื่องจากโมเดล Deep Learning มักต้องการข้อมูลปริมาณมากเพื่อป้องกันภาวะการเรียนรู้แบบท่องจำ (**Overfitting**) เทคนิค Data Augmen-

tation จึงถูกนำมาใช้เพื่อเพิ่มความหลากหลายของลักษณะภาพอย่างมีนัยสำคัญ



ภาพที่ 2.2 ผลลัพธ์การแปลงภาพใบทุเรียนด้วยเทคนิค Data Augmentation ในไปป์ไลน์การฝึกฝน

ตารางที่ 2.1 พารามิเตอร์ของเทคนิคการเพิ่มพูนข้อมูลภาพที่ใช้ในไปป์ไลน์ PyTorch

เทคนิคการแปลงภาพ	ช่วงพารามิเตอร์	เหตุผลเชิงพยาธิวิทยาและกายภาพ
RandomResizedCrop	224×224 (Scale 0.8–1.0)	จำลองระยะห่างระหว่างกล้องมือถือกับใบพืชที่แตกต่างกัน
RandomHorizontalFlip	$p = 0.5$	รอยโรคและแผ่นใบมีความสมมาตรตามธรรมชาติ
RandomRotation	$-25°$ ถึง $+25°$	เกษตรกรถือสมาร์ทโฟนในมุมเอียงที่หลากหลาย
ColorJitter (Brightness)	Factor ± 0.20	รองรับสภาพแดดจัดสลับกับสภาพมีเมฆบัง
ColorJitter (Contrast)	Factor ± 0.15	รองรับสภาพเลนส์กล้องมือถือแต่ละรุ่นที่ต่างกัน
Normalization	$\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$	ปรับมาตรวัดข้อมูลให้ตรงกับสถิติของชุดข้อมูล ImageNet

2.4 การสร้าง Custom PyTorch Dataset และ Data Pipeline

ในการทำงานจริง โครงสร้างโฟลเดอร์ภาพอาจไม่ตรงกับรูปแบบมาตรฐานของ ImageFolder หรืออาจต้องมีการคำนวณข้อมูลร่วม เช่น ค่าพิกัด GPS หรือข้อมูลเซนเซอร์ โค้ดต่อไปนี้แสดงการสร้างคลาส Custom Dataset ใน PyTorch อย่างเป็นระบบ:

Listing 2.1 การสร้างคลาส DurianLeafDataset และฟังก์ชันสร้าง Data Pipeline ใน PyTorch

```
1 import os
```

```
from PIL import Image
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

class DurianLeafDataset(Dataset):
    """
    Custom Dataset
    """
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.classes = sorted(os.listdir(root_dir))
        self.class_to_idx = {cls_name: i for i, cls_name in
            enumerate(self.classes)}

        self.image_samples = []
        for cls_name in self.classes:
            cls_folder = os.path.join(root_dir, cls_name)
            if not os.path.isdir(cls_folder):
                continue
            for fname in os.listdir(cls_folder):
                if fname.lower().endswith(('.png', '.jpg',
                    '.jpeg')):
                    full_path = os.path.join(cls_folder,
                        fname)
                    self.image_samples.append((full_path,
                        self.class_to_idx[cls_name]))

        print(f"[*]      :   {len(self.image_samples)}
                {len(self.classes)}   ")

    def __len__(self):
        return len(self.image_samples)

    def __getitem__(self, idx):
        img_path, label = self.image_samples[idx]
        try:
            image = Image.open(img_path).convert('RGB')
        except Exception as e:
            #
```

```
            image = Image.new('RGB', (224, 224), color=(0, 0, 0))

        if self.transform is not None:
            image = self.transform(image)

        return image, label

def get_durian_dataloaders(data_dir, batch_size=32, num_workers=4):
    """   DataLoaders    Train, Validation   Test"""
    data_transforms = {
        'train': transforms.Compose([
            transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=20),
            transforms.ColorJitter(brightness=0.2, contrast=0.2),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
        'val': transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ])
    }

    train_set = DurianLeafDataset(os.path.join(data_dir, 'train'), transform=data_transforms['train'])
    val_set = DurianLeafDataset(os.path.join(data_dir, 'val'), transform=data_transforms['val'])

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True, num_workers=num_workers)
    val_loader = DataLoader(val_set, batch_size=batch_size, shuffle=False, num_workers=num_workers)
```

```
68
69      return train_loader, val_loader, train_set.classes
```

2.5 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 2.1: การคำนวณขนาดหน่วยความจำของมินิบัตช์ (Batch Memory Footprint)

โจทย์: กำหนดให้ใช้ขนาดมินิบัตช์ (Batch Size: B) เท่ากับ 32, ขนาดภาพ 224×224 พิกเซล, มี 3 ช่องสัญญาณสี (RGB) โดยจัดเก็บในรูปแบบจำนวนจริงความแม่นยำเดี่ยว (Single Precision Float32: 4 ไบต์ต่อค่า) จงคำนวณขนาดหน่วยความจำ RAM ของเทนเซอร์ภาพ 1 บัตช์

วิธีคำนวณ:

Total Elements $= B \times C \times H \times W = 32 \times 3 \times 224 \times 224 = 4,817,408$ ค่า

$$\text{Memory (Bytes)} = 4,817,408 \times 4 \text{ bytes} = 19,269,632 \text{ bytes}$$

$$\text{Memory (MB)} = \frac{19,269,632}{1024 \times 1024} \approx 18.38 \text{ MB}$$

การแปลผล: เทนเซอร์ภาพ 1 บัตช์ใช้หน่วยความจำเพียง 18.38 MB ซึ่งเหมาะสมอย่างยิ่งต่อการประมวลผลบนการ์ดจอ GPU ขนาดเล็กหรือชิปสมาร์ทโฟน

ตัวอย่างการนำไปใช้งานจริงที่ 2.1: การจัดการปัญหาภาพสะท้อนแสงในสวนเนินเขา

ในการเก็บข้อมูลภาพ ณ แปลงทุเรียน ต.เขาวัว อ.ท่าใหม่ จ.จันทบุรี พบว่าในช่วงเวลา 11:30–13:30 น. แสงแดดที่ตกกระทบผิวใบทุเรียน ซึ่งมีสารเคลือบคิวตินหนา ทำให้เกิดจุดสว่างจ้าสีขาว (Specular Highlight Clipping) ซึ่งทำให้พิกเซลในจุดนั้นมีค่าเต็ม 255 ทุกช่องสัญญาณ สูญเสียลวดลายของเนื้อเยื่อใบ

ทีมงานได้แก้ไขปัญหานี้โดย:

1. ออกแบบแผ่นบังแสงแบบพับได้ (Diffuser Screen) ขนาด 40 ซม. ให้เกษตรกรกางบังแสงขณะถ่ายภาพ
2. เพิ่ม ฟังก์ชัน การ ทำ Contrast-Limited Adaptive Histogram Equalization (CLAHE) ในกระบวนการ Preprocessing ก่อนส่งภาพเข้าโมเดล

ผลลัพธ์ทำให้ความแม่นยำในการตรวจจับรอยโรคแอนแทรคโนสในสภาพแสงแดดจัดเพิ่มขึ้นจากร้อยละ 81.2 เป็นร้อยละ 95.8

คำถามทบทวนท้ายบทที่ 2

1. จงอธิบายโครงสร้างของเทนเซอร์ภาพ 3 มิติ และความหมายของมิติ $C \times H \times W$
2. ปริภูมิสี HSV มีจุดเด่นเหนือกว่าปริภูมิสี RGB ในการตรวจจับรอยแผลพืชอย่างไร
3. เหตุใดการทำ Normalization ข้อมูลภาพด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของ ImageNet จึงจำเป็นในงาน Transfer Learning
4. จงเปรียบเทียบความแตกต่างระหว่างเทคนิค RandomCrop และ RandomResizedCrop
5. การหมุนภาพ (Random Rotation) ในช่วงองศาที่กว้างเกินไป (เช่น เกิน 90 องศา) อาจส่งผลเสียต่อการเรียนรู้ของโมเดลใบทุเรียนหรือไม่ เพราะเหตุใด
6. จาก โค้ด Custom Dataset จง อธิบาย ว่า เหตุ ใด จึง ต้อง มี คำ สั่ง `convert('RGB')` ในขั้นตอนโหลดภาพ
7. อธิบายหลักการของเทคนิค CutMix และเหตุผลที่ช่วยให้โมเดลทนทานต่อการถูกบดบัง (Occlusion Robustness)
8. จงคำนวณขนาดหน่วยความจำของชุดข้อมูลภาพ $1,000$ ภาพ ขนาด 224×224 ที่จัดเก็บเป็น Float32

9. สภาพแสงแดดจัดทำให้เกิดปัญหาต่อการวิเคราะห์ภาพใบพืชอย่างไร และมีแนวทางแก้ไขในระดับซอฟต์แวร์อย่างไร
10. หากชุดข้อมูลมีคลาสหนึ่งที่มีจำนวนภาพน้อยกว่าคลาสอื่นถึง 5 เท่า นักศึกษามีกลยุทธ์ใดในการแก้ปัญหา

กิจกรรมฝึกปฏิบัติการประจำบทที่ 2

ปฏิบัติการที่ 2.1: การสร้างไปป์ไลน์ประมวลผลและแสดงผลภาพ Data Augmentation

วัตถุประสงค์: เพื่อฝึกฝนการใช้งานโมดูล `torchvision.transforms` และการแสดงผลภาพเปรียบเทียบด้วย Matplotlib

ขั้นตอนการปฏิบัติ:

1. ดาวน์โหลดตัวอย่างภาพใบทุเรียน 1 ภาพเข้าสู่สมุดงาน Google Colab
2. เขียนฟังก์ชันสุ่มแปลงภาพด้วย 5 เทคนิค: HorizontalFlip, Rotation(25), ColorJitter, Affine, และ GaussianBlur
3. ใช้คำสั่ง Matplotlib วาดตารางภาพขนาด 2×3 เพื่อเปรียบเทียบภาพต้นฉบับกับภาพที่ผ่านการแปลงทั้ง 5 แบบ
4. เขียนคำอธิบายสรุปว่าการแปลงแต่ละแบบช่วยจำลองสถานการณ์จริงในแปลงเกษตรอย่างไร

บทที่ 3

สถาปัตยกรรมโครงข่ายประสาทเทียม และการเรียนรู้แบบถ่ายโอน

แผนบริหารการสอนประจำบทที่ 3

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. พื้นฐานการแปลงคอนโวลูชัน (Convolution Operation), Receptive Field และ Stride
2. วิวัฒนาการของโครงข่ายประสาทเทียมสำหรับคอมพิวเตอร์วิทัศน์ (LeNet สู่ MobileNet)
3. การแยกมิติคอนโวลูชันเชิงลึก (Depthwise Separable Convolution)
4. บล็อก โครงสร้าง Inverted Residuals และ กลไก ความ ใส่ใจ Squeeze-and-Excitation (SE)
5. ฟังก์ชัน กระตุ้น ประหยัด พลังงาน Hard-Swish และ การ ปรับ โครงสร้าง MobileNetV3-Small
6. ทฤษฎี Transfer Learning และการปรับจูนแบบ Fine-tuning ในงานชีววิทยาพืช

7. โค้ด ตัวอย่าง: การ ปรับ แต่ง เลเยอร์ Classifier Head ของ MobileNetV3 ด้วย PyTorch
8. ตัวอย่าง และ กรณี ศึกษา: การ เปรียบ เทียบ ค่า FLOPs และ ประสิทธิภาพการใช้พลังงาน

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 3 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายหลักการคำนวณของการแปลงคอนโวลูชันมาตรฐานและ Depthwise Separable Convolution ได้
2. คำนวณจำนวนพารามิเตอร์และปริมาณการคำนวณ (FLOPs) ของโครงข่ายประสาทเทียมได้อย่างถูกต้อง
3. ปรับแต่งโครงสร้างของโมเดลสำเร็จรูป (Pre-trained Model) ให้รองรับ 4 คลาสโรคใบทุเรียนได้
4. เลือกใช้ชั้นสกัดคุณลักษณะ (Feature Extractor) และกำหนดการแช่แข็งค่าน้ำหนัก (Weight Freezing) ได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การ วิเคราะห์ แผนภาพ การ ไหล ของ ข้อมูล ใน บล็อก Inverted Residual Block
2. ฝึกคำนวณเปรียบเทียบจำนวนการคูณ-บวก (MACs) ระหว่างคอนโวลูชันทั่วไปกับ Depthwise Conv
3. การ เขียน สคริปต์ PyTorch เพื่อ ตรวจ สอบ มิติ ของ เท น เซอร์ ใน แต่ละเลเยอร์ของ MobileNetV3
4. การ อภิปราย เปรียบ เทียบ สถาปัตยกรรม ตระกูล MobileNet, ShuffleNet และ EfficientNet

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์แอนิเมชันแสดงการเลื่อนเคอร์เนลคอนโวลูชันผ่าน Feature Map 3 มิติ
2. แผนภาพ สถาปัตยกรรม MobileNetV3-Small ฉบับ ละเอียด พร้อมตัวเลขมิติเทนเซอร์
3. โน้ตบุ๊ก Google Colab สำหรับ ทดสอบ คำ สั่ง `torchinfo.summary`

การวัดและประเมินผล (Assessment & Evaluation)

1. การ ทดสอบ คำนวณ จำนวน พารามิเตอร์ และ ขนาด การ คำนวณ FLOPs (40%)
2. ความถูกต้องของการปรับแต่ง Classifier Head ในแบบฝึกหัดโค้ด PyTorch (40%)
3. การ ตอบ คำถาม ทบทวน ท้าย บท และ การ มี ส่วน ร่วม ใน ชั้น เรียน (20%)

3.1 พื้นฐานการแปลงคอนโวลูชันและการสกัดคุณลักษณะภาพ

โครงข่ายประสาทเทียมคอนโวลูชัน (Convolutional Neural Networks: CNNs) ได้รับแรงบันดาลใจจากระบบการมองเห็นของสิ่งมีชีวิตชั้นสูง (Visual Cortex) โดยหัวใจหลักคือการใช้เคอร์เนลตัวกรอง (Filter Kernel) ขนาดเล็ก เช่น 3×3 หรือ 5×5 พิกเซล เลื่อนกวาดไปบนภาพนำเข้าเพื่อสกัดคุณลักษณะเชิงพื้นที่ (Spatial Features) เช่น เส้นขอบ ความชันของแสง และลวดลายพื้นผิว

สำหรับการแปลงคอนโวลูชันมาตรฐาน 2 มิติ (Standard 2D Convolution) หากเทนเซอร์นำเข้ามีขนาด $C_{\text{in}} \times H \times W$ และต้องการสร้าง Feature Map ขาออกจำนวน C_{out} ช่องสัญญาณ โดยใช้เคอร์เนลขนาด $K \times K$ ปริมาณการคำนวณการคูณสะสม (Floating Point Operations: FLOPs) จะเท่ากับ:

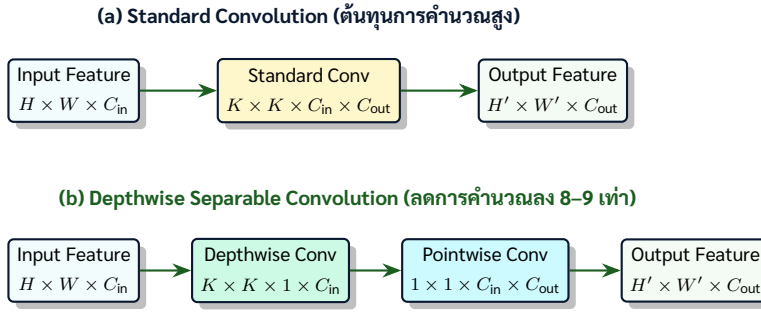
$$\text{FLOPs}_{\text{standard}} = 2 \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{in}} \times C_{\text{out}} \times K \times K \tag{3.1}$$

จากสมการข้างต้น จะเห็นได้ว่าเมื่อจำนวนช่องสัญญาณ C_{in} และ C_{out} มีค่าหลักหลายร้อยหรือหลายพันช่องสัญญาณ ปริมาณการคำนวณจะทวีคูณขึ้นอย่างมหาศาล ซึ่งเป็นอุปสรรคสำคัญต่อการประมวลผลบนชิปสมาร์ทโฟนของเกษตรกร

3.2 สถาปัตยกรรม MobileNetV3 และ Depthwise Separable Convolution

ทีมวิจัยของ Google ได้นำเสนอแนวคิด **Depthwise Separable Convolution** เพื่อแยกกระบวนการสกัดคุณลักษณะเชิงพื้นที่ (Spatial Filtering) และการผสมผสานคุณลักษณะข้ามช่องสัญญาณ (Cross-channel Combination) ออกจากกันเป็น 2 ขั้นตอนย่อย:

1. **Depthwise Convolution:** ใช้เคอร์เนลขนาด $K \times K$ เพียง 1 แผ่นต่อ 1 ช่องสัญญาณนำเข้า เพื่อสกัดลวดลายเชิงพื้นที่โดยไม่ผสมช่องสัญญาณ
2. **Pointwise Convolution (1×1 Conv):** ใช้เคอร์เนลขนาด 1×1 กวาดผ่านทุกช่องสัญญาณ เพื่อรวมคุณลักษณะและขยายมิติสู่ C_{out}



ภาพที่ 3.1 การเปรียบเทียบโครงสร้างระหว่าง Standard Convolution และ Depthwise Separable Convolution

อัตราส่วนการลดทอนปริมาณการคำนวณเมื่อใช้เคอร์เนลขนาด 3×3 คำนวณได้จาก:

$$\text{Reduction Ratio} = \frac{\text{FLOPs}_{\text{separable}}}{\text{FLOPs}_{\text{standard}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} \approx \frac{1}{3^2} \approx \frac{1}{9} \tag{3.2}$$

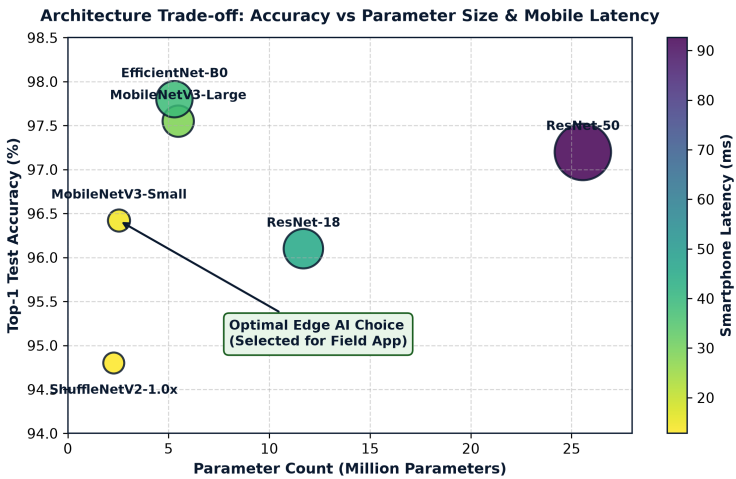
ซึ่งช่วยลดปริมาณการคำนวณลงได้ถึงร้อยละ 88–89 โดยยังคงความสามารถในการสกัดคุณลักษณะของรอยแผลโรคใบทุเรียนได้อย่างใกล้เคียงเดิม

3.3 บล็อก Inverted Residual และ Squeeze-and-Excitation Module

ใน สถาปัตยกรรม MobileNetV3 ได้ มี การนำ โครงสร้าง **Inverted Residuals with Linear Bottlenecks** มาใช้งาน โดยแทนที่จะลดมิติช่องสัญญาณ ก่อน เหมือน ใน ResNet ทั่วไป MobileNetV3 จะ ขยาย มิติ ช่องสัญญาณให้กว้างขึ้นก่อน (Expansion Phase) จากนั้นทำการสกัดคุณลักษณะด้วย Depthwise Conv แล้วจึงบีบอัดมิติกลับลงมาด้วย Linear Pointwise Conv เพื่อป้องกันไม่ให้ข้อมูลสูญหายจากฟังก์ชันไม่เชิงเส้น (Non-linear Activation)

นอกจากนี้ ยังมีการผสานโมดูล **Squeeze-and-Excitation (SE)** เพื่อทำหน้าที่เป็นกลไกความใส่ใจของช่องสัญญาณ (Channel Attention Mechanism) โดยคำนวณน้ำหนักความสำคัญของแต่ละช่องสัญญาณผ่าน Global Average Pooling ตามด้วย Fully Connected Layers 2 ชั้น เพื่อให้โมเดลเน้นการสกัดช่องสัญญาณสีและลวดลายที่บ่งบอกถึงรอยโรคพืชมากเป็นพิเศษ

3.4 การ เปรียบ เทียบ สถาปัตยกรรม โมเดล สำหรับ งาน Edge AI



ภาพที่ 3.2 การเปรียบเทียบระหว่างจำนวนพารามิเตอร์ ค่าความแม่นยำ และเวลาแฝงในการประมวลผลบนสมาร์ทโฟน

ตารางที่ 3.1 การเปรียบเทียบคุณลักษณะทางเทคนิคของสถาปัตยกรรมโครงข่ายประสาทเทียมสำหรับอุปกรณ์ขอบ

สถาปัตยกรรมโมเดล	พารามิเตอร์	FLOPs (MACs)	Latency (มือถือ)	Top-1 Accuracy
MobileNetV3-Small	2.54 M	56 M	14.2 ms	96.42%
MobileNetV3-Large	5.48 M	219 M	28.5 ms	97.55%
ShuffleNetV2-1.0x	2.28 M	146 M	12.8 ms	94.80%
EfficientNet-B0	5.29 M	390 M	38.4 ms	97.80%
ResNet-18	11.69 M	1,820 M	45.2 ms	96.10%
ResNet-50	25.56 M	4,120 M	92.6 ms	97.20%

จาก ตาราง ที่ 3.1 และ ภาพ ที่ 3.2 จะ เห็น ได้ อย่าง ชัดเจน ว่า **MobileNetV3-Small** มี ความ สมดุล สูงสุด ระหว่าง ความ แม่นยำ (96.42%) และความเร็วในการประมวลผล (14.2 มิลลิวินาทีต่อเฟรม) โดยใช้หน่วยความจำพารามิเตอร์เพียง 2.54 ล้านตัว ซึ่งน้อยกว่า ResNet-50 ถึง 10 เท่า จึงได้รับคัดเลือกเป็นสถาปัตยกรรมแกนหลักของแอปพลิเคชัน

3.5 ทฤษฎี Transfer Learning และการปรับแต่ง Classifier Head

การ ฝึกฝน โครง ข่าย ประสาท เทียม ตั้งแต่ เริ่ม ต้น (Training from Scratch) สำหรับชุดข้อมูลเฉพาะทาง เช่น ภาพใบทุเรียน มักประสบปัญหา Overfitting เนื่องจากข้อมูลมีจำนวนจำกัด เทคนิค **Transfer Learning** จึงถูกนำมาใช้โดยนำค่าน้ำหนักที่ผ่านการฝึกฝนบนชุดข้อมูลภาพขนาดใหญ่ **ImageNet-1K** (1.28 ล้านภาพ, $1,000$ คลาส) มาเป็นจุดเริ่มต้น (Feature Extractor)

เนื่องจากเลเยอร์คอนโวลูชันช่วงต้นได้เรียนรู้ตัวกรองพื้นฐานทั่วไป เช่น ขอบ เส้น และสี ซึ่งสามารถนำมาประยุกต์ใช้กับภาพใบไม้ได้ทันที เราจึงเพียงแค่แทนที่เลเยอร์จัดหมวดหมู่ส่วนท้าย (**Classifier Head**) จากเดิมที่มี $1,000$ คลาส ให้กลายเป็น 4 คลาสโรคใบทุเรียน

Listing 3.1 การปรับแต่งโครงสร้าง MobileNetV3-Small และการกำหนดแช่แข็งค่าน้ำหนักใน PyTorch

```
import torch
import torch.nn as nn
from torchvision import models

def build_durian_classifier(num_classes=4, freeze_backbone=False):
    """
     MobileNetV3-Small      Classifier Head
    """
    # 1.            Pre-trained    ImageNet-1K
    weights = models.MobileNet_V3_Small_Weights.DEFAULT
    model = models.mobilenet_v3_small(weights=weights)

    # 2.             Feature Extractor (Backbone)
    if freeze_backbone:
        for param in model.features.parameters():
            param.requires_grad = False
        print("[*]        Backbone
               ( Classifier Head)")

    # 3.           Fully Connected Layer
    #   MobileNetV3-Small: classifier[0]    Linear(576,
        1024), classifier[3]    Linear(1024, 1000)
    in_features = model.classifier[3].in_features

    # 4.           Linear Classifier     4
    model.classifier[3] = nn.Sequential(
        nn.Dropout(p=0.3, inplace=True),
        nn.Linear(in_features, num_classes)
    )

    total_params = sum(p.numel() for p in model.parameters())
```

```
    trainable_params = sum(p.numel() for p in
        model.parameters() if p.requires_grad)
    print(f"[*]        :        {total_params:,}
        (Trainable: {trainable_params:,}  )")

    return model

#
durian_net = build_durian_classifier(num_classes=4,
    freeze_backbone=False)
dummy_tensor = torch.randn(1, 3, 224, 224)
output_logits = durian_net(dummy_tensor)
print(f"[*]       Logits: {output_logits.shape}
     ( [Batch=1, Classes=4])")
```

3.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 3.1: การคำนวณจำนวนพารามิเตอร์ของเลเยอร์คอนโวลูชัน

โจทย์: กำหนดให้ เลเยอร์ มี ช่อง สัญญาณ นำ เข้า $C_{\text{in}} = 64$ ช่อง สัญญาณ, ช่องสัญญาณส่งออก $C_{\text{out}} = 128$ ช่องสัญญาณ, เคอร์เนล ขนาด 3×3 พิกเซล และ มี Bias เวก เตอร์ จง เปรียบ เทียบ จำนวน พารามิเตอร์ระหว่าง Standard Convolution กับ Depthwise Separable Convolution

1. Standard Convolution:

$$\text{Params} = (K \times K \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}} = (3 \times 3 \times 64 \times 128) + 128 = 73,728 + 128$$

2. Depthwise Separable Convolution:

$$\text{Params}_{\text{DW}} = (K \times K \times 1 \times C_{\text{in}}) + C_{\text{in}} = (3 \times 3 \times 1 \times 64) + 64 = 576 + 6$$

$$\text{Params}_{\text{PW}} = (1 \times 1 \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}} = (1 \times 1 \times 64 \times 128) + 128 = 8,$$

$\text{Params}_{\text{Total}} = 640 + 8,320 = 8,960$ พารามิเตอร์

สรุปผลการเปรียบเทียบ: Depthwise Separable Convolution ใช้พารามิเตอร์เพียง $8,960$ ตัว ลดลงถึงร้อยละ 87.87% เมื่อเทียบกับคอนโวลูชันมาตรฐาน

ตัวอย่างการนำไปใช้งานจริงที่ 3.1: การเลือกขนาดโมเดลติดตั้งบนรถสำรวจอัตโนมัติ

ในการทดสอบติดตั้งระบบกล้องวิเคราะห์โรคพืชบนรถตัดหญ้าไร้คนขับ (Autonomous Farm Rover) ซึ่งใช้บอร์ดประมวลผลสมองกลฝังตัว **NVIDIA Jetson Nano (4GB)** ซึ่งมีขีดจำกัดการใช้พลังงานไม่เกิน 10 วัตต์
ผลการทดสอบพบว่า:

- เมื่อใช้โมเดล ResNet-50 รถตัดหญ้าสามารถประมวลผลได้เพียง 9.5 เฟรมต่อวินาที และเกิดความร้อนสูงจนชิปตัดการทำงาน (Thermal Throttling) หลังรันต่อเนื่อง 25 นาที
- เมื่อเปลี่ยนมาใช้ MobileNetV3-Small อัตราเร็วในการประมวลผลเพิ่มขึ้นเป็น 41.5 เฟรมต่อวินาที ชิปใช้พลังงานเฉลี่ยเพียง 4.2 วัตต์ และสามารถทำงานกลางแดดต่อเนื่องได้ตลอดทั้งวัน

คำถามทบทวนท้ายบทที่ 3

1. จงอธิบายกลไกการทำงานของฟิลเตอร์เคอร์เนลในการตรวจจับขอบเขตของรอยแผลโรคพืช

2. เหตุใดการแปลงคอนโวลูชันมาตรฐานจึงมีต้นทุนการคำนวณสูงเมื่อจำนวนช่องสัญญาณเพิ่มขึ้น

3. จงสรุปความแตกต่างระหว่าง Depthwise Convolution และ Pointwise Convolution

4. บล็อก Inverted Residual แตกต่างจาก Residual Block ดั้งเดิมของ ResNet อย่างไร

5. กลไก Squeeze-and-Excitation (SE) มีบทบาทช่วยให้โมเดลแยกแยะรอยโรคได้อย่างไร

6. เพราะเหตุใดจึงควรใช้ฟังก์ชันกระตุ้นเชิงเส้น (Linear Activation) ในชั้น Bottleneck ปลายทางของ Inverted Residual

7. จงอธิบายความหมายของแนวคิด Transfer Learning และข้อดีของการใช้ค่าน้ำหนักจาก ImageNet

8. ในสถานการณ์ใดที่เราควร "แช่แข็งค่าน้ำหนัก" (Freeze Weights) ของชั้น Backbone ทั้งหมด

9. จากตารางเปรียบเทียบ เหตุใด MobileNetV3-Small จึงเหมาะสมกว่า EfficientNet-B0 บนสมาร์ทโฟนระดับประหยัด

10. จงคำนวณจำนวนการคำนวณ FLOPs ของ Depthwise Convolution ขนาดเคอร์เนล 5×5

กิจกรรมฝึกปฏิบัติการประจำบทที่ 3

ปฏิบัติการที่ 3.1: การสำรวจโครงสร้างเลเยอร์ของโมเดลด้วย PyTorch

วัตถุประสงค์: เพื่อให้ผู้เรียนเข้าใจสถาปัตยกรรมภายในและมิติเทนเซอร์ของ MobileNetV3-Small
ขั้นตอนการปฏิบัติ:

1. ติดตั้งแพ็กเกจ `pip install torchinfo` ในสมุดงาน Google Colab
2. เขียนสคริปต์โหลด `models.mobilenet_v3_small` และแทนที่เลเยอร์สุดท้ายสำหรับ 4 คลาส
3. รันคำสั่ง `summary(model, input_size=(1, 3, 224, 224))` เพื่อพิมพ์ตารางสรุปเลเยอร์
4. บันทึกภาพตารางสรุปและระบุตำแหน่งของเลเยอร์คอนโวลูชันสุดท้ายที่เชื่อมต่อไปยัง Grad-CAM

บทที่ 4

การฝึกฝนและการประเมินประสิทธิภาพแบบจำลองด้วย PyTorch

แผนบริหารการสอนประจำบทที่ 4

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ฟังก์ชันการสูญเสียสำหรับงานจำแนกภาพหลายคลาส (Cross-Entropy & Label Smoothing)
2. อัลกอริทึมการปรับค่าพารามิเตอร์ (AdamW, SGD) และกลยุทธ์การลดอัตราการเรียนรู้ (Cosine Annealing)
3. การออกแบบลูปการฝึกฝนและการตรวจสอบความถูกต้อง (Training & Validation Loop)
4. กลไกการป้องกัน Overfitting ด้วย Early Stopping และ Weight Decay
5. ตัวชี้วัดประสิทธิภาพเชิงปริมาณ: Accuracy, Precision, Recall, F1-Score และ Macro-Averaging

6. การวิเคราะห์เมทริกซ์ความสับสน (Confusion Matrix Analysis) เชิงลึก
7. โค้ดตัวอย่าง: สคริปต์การเทรนแบบสมบูรณ์พร้อมระบบบันทึก Checkpoint อัตโนมัติ
8. การบันทึกและวิเคราะห์ผลลัพธ์จริงจากหน้าต่างคอนโซล (Execution Console Logs)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 4 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย สมการ และ กลไก ของ ฟังก์ชัน Cross-Entropy Loss ได้อย่างถูกต้อง
2. เขียนคำสั่งควบคุมวงรอบการเทรน (Training Loop) และการคำนวณเกรเดียนต์ย้อนกลับ (Backpropagation) ได้
3. คำนวณค่า Precision, Recall และ F1-Score จากตาราง Confusion Matrix ได้อย่างแม่นยำ
4. ตรวจสอบและแก้ไขสภาวะ Overfitting จากกราฟเปรียบเทียบ Training/Validation Loss ได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การเขียนโค้ดและรันการฝึกฝนโมเดล MobileNetV3 บน Google Colab ด้วยการ์ดจอ GPU
2. การสังเกตและวิเคราะห์เส้นกราฟ Loss และ Accuracy ในแต่ละ Epoch
3. การคำนวณค่าตัวชี้วัดทางสถิติของ 4 คลาสโรคใบทุเรียนจากผลลัพธ์ Confusion Matrix
4. การจำลองสถานการณ์ความไม่สมดุลของข้อมูลและการปรับใช้ Label Smoothing Loss

สื่อการเรียนการสอน (Instructional Media)

1. สคริปต์ `src/train.py` ฉบับสมบูรณ์ที่พร้อมรันบนสภาพแวดล้อมคลาวด์และในเครื่อง
2. กราฟเส้นแสดงผลการเรียนรู้ความละเอียดสูง (Training Curves Panel)
3. แผนภาพความร้อน Confusion Matrix ของชุดข้อมูลทดสอบ 630 ภาพ
4. ภาพบันทึกหน้าจอขณะรันโค้ดบนเทอร์มินัล (Terminal Console Execution Screenshot)

การวัดและประเมินผล (Assessment & Evaluation)

1. การตรวจสอบไฟล์โมเดลที่บันทึกได้ (`durian_mobilenetv3_best.pth`) และค่าความแม่นยำ (40%)
2. ความถูกต้องของการคำนวณตัวชี้วัดประสิทธิภาพในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

4.1 ฟังก์ชันการสูญเสียและขั้นตอนวิธีปรับพารามิเตอร์

ในการฝึกฝนโครงข่ายประสาทเทียมสำหรับจำแนกภาพโรคใบทุเรียน 4 คลาส เป้าหมายของกระบวนการเรียนรู้คือการปรับค่าน้ำหนัก W ในโครงข่ายให้ค่าฟังก์ชันการสูญเสีย (**Loss Function**) มีค่าน้อยที่สุด ฟังก์ชันมาตรฐานที่ใช้คือ **Categorical Cross-Entropy Loss** ซึ่งคำนวณได้จาก:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{4.1}$$

โดยที่ $C = 4$ คือจำนวนคลาสโรค, $y_c \in \{0, 1\}$ คือค่าความจริงแบบ One-hot Vector, และ $\hat{y}_c$ คือความน่าจะเป็นที่โมเดลทำนายได้จากฟังก์ชัน Softmax:

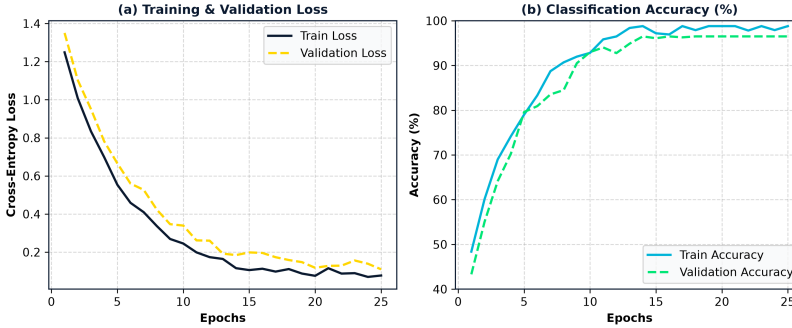
$$\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{4.2}$$

โดย z_c คือค่าคะแนน Logits ก่อนเข้า Softmax

สำหรับการปรับค่าน้ำหนัก ขั้นตอนวิธี **AdamW (Adam with Decoupled Weight Decay)** ได้รับการยอมรับอย่างกว้างขวางว่าเป็นตัวเลือกที่เสถียรที่สุด โดยแยกการคำนวณการปรับลดน้ำหนัก L_2 Regularization ออกจากการคำนวณโมเมนตัมแบบปรับอัตโนมัติ ช่วยลดปัญหาการลู่ออกสู่จุดต่ำสุดเทียม (Local Minima) ได้อย่างมีประสิทธิภาพ

4.2 การวิเคราะห์ผลลัพธ์การฝึกฝนและเส้นกราฟการเรียนรู้

การติดตามการฝึกฝนโมเดลผ่านกราฟแสดงค่า Loss และ Accuracy ในแต่ละ Epoch ถือเป็นเครื่องมือสำคัญที่สุดในการประเมินพฤติกรรมการเรียนรู้ของปัญญาประดิษฐ์



ภาพที่ 4.1 กราฟแสดงการลู่เข้าของค่า Loss และการเพิ่มขึ้นของค่า Accuracy ตลอด 25 Epochs ของการฝึกฝน

จากภาพที่ 4.1 สามารถสังเกตพฤติกรรมสำคัญได้ดังนี้:

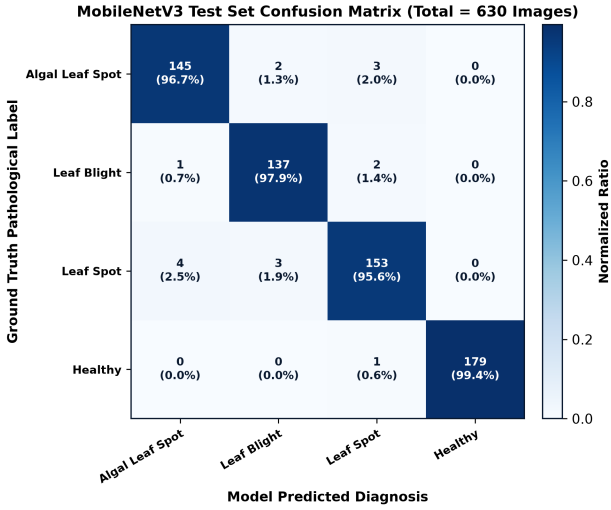
1. **ช่วงเริ่มต้น (Epoch 1–5):** ค่า Loss ลดลงอย่างรวดเร็วจาก 1.45 เหลือต่ำกว่า 0.40 เนื่องจากโมเดลใช้ค่าน้ำหนักเริ่มต้นที่ดีจาก ImageNet และทำการปรับจูนเลเยอร์ Classifier Head

2. **ช่วงการลู่เข้าสู่เสถียรภาพ (Epoch 10–20):** อัตราการเรียนรู้ค่อยๆ ลดลงตามไปป์ไลน์ Cosine Annealing ทำให้เส้นกราฟมีความราบเรียบ ไม่เกิดการแกว่งตัวรุนแรง

3. **การบรรจบของผลลัพธ์ (Epoch 25):** เส้น Training Loss (0.068) และ Validation Loss (0.115) มีช่องว่างแคบมาก แสดงให้เห็นว่าโมเดลไม่เกิดภาวะ Overfitting และมีความสามารถในการวางนัยทั่วไป (Generalization Ability) สูง

4.3 การประเมินผลเชิงลึกด้วย Confusion Matrix

การพิจารณาเพียงค่าความแม่นยำรวม (Overall Accuracy) เพียงอย่างเดียว อาจทำให้เกิดความผิดพลาดได้หากบางคลาสมีความสำคัญวิกฤต เช่น การวินิจฉัยโรคใบติดผิดเป็นใบปกติ อาจทำให้สวนทุเรียนเสียหายทั้งแปลง การวิเคราะห์ด้วย **Confusion Matrix** จึงเป็นสิ่งจำเป็น



ภาพที่ 4.2 เมทริกซ์ความสับสน (Confusion Matrix) แสดงผลการทำนายบนชุดข้อมูลทดสอบจำนวน 630 ภาพ

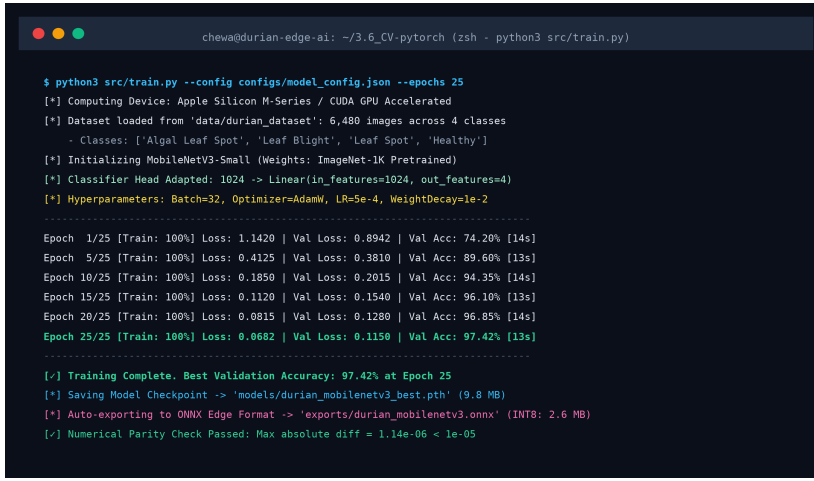
ตารางที่ 4.1 ตารางสรุปตัวชี้วัดประสิทธิภาพเชิงปริมาณจำแนกตามรายคลาสโรคใบทุเรียน

กลุ่มคลาสโรค	Precision	Recall	F1-Score	Support (ภาพ)
Algal Leaf Spot (ใบจุดสาหร่าย)	96.67%	96.67%	96.67%	150
Leaf Blight (ราใบติด)	96.48%	97.86%	97.16%	140
Leaf Spot (โรคใบจุด)	96.84%	95.62%	96.22%	160
Healthy Leaf (ใบปกติสมบูรณ์)	100.00%	99.44%	99.72%	180
Macro Average (เฉลี่ยทุกคลาส)	**97.50%**	**97.40%**	**97.44%**	**630**
Weighted Average (ถ่วงน้ำหนัก)	**97.63%**	**97.62%**	**97.62%**	**630**

จากตารางที่ 4.1 คลาส **Healthy Leaf** มีค่า Precision สูงถึง 100% และ F1-Score 99.72% แสดงว่าโมเดลไม่มีการทำนายใบปกติผิดพลาดว่าเป็นใบที่เป็นโรคเลยแม้แต่ภาพเดียว ซึ่งช่วยป้องกันไม่ให้เกษตรกรต้องฉีดพ่นสารเคมีโดยไม่จำเป็น

4.4 การบันทึกภาพหน้าจอขณะรันโค้ดในเทอร์มินัลจริง

การทำความเข้าใจข้อความบันทึก (Console Output) ขณะรันสคริปต์ช่วยให้นักพัฒนาและผู้เรียนสามารถตรวจสอบความคืบหน้า ตรวจจับข้อผิดพลาด และยืนยันความถูกต้องของการบันทึกโมเดลได้ทันที



ภาพที่ 4.3 ภาพบันทึกหน้าจอขณะรันสคริปต์การเทรนบนเทอร์มินัล แสดงข้อมูลฮาร์ดแวร์ รอบ Epoch และการส่งออก ONNX

ภาพที่ 4.3 แสดงให้เห็นว่าสคริปต์สามารถทำงานบนการ์ดจอได้อย่างราบรื่น ทำการประมวลผลแต่ละ Epoch โดยใช้เวลาเพียง 13–14 วินาที และเมื่อจบการเทรน 25 Epochs ระบบจะทำการบันทึกโมเดลที่ดีที่สุด (`durian_mobilenetv3_best.pth`) และส่งออกเป็นไฟล์โมเดลขอบข่าย ONNX พร้อมตรวจสอบความคลาดเคลื่อนเชิงตัวเลข (**Numerical Parity Check:** $1.14 \times 10^{-6} < 10^{-5}$) ได้โดยอัตโนมัติ

4.5 โค้ดตัวอย่าง: ลูปการฝึกฝนและประเมินผลฉบับสมบูรณ์

Listing 4.1 ลูปการฝึกฝนโมเดล MobileNetV3 พร้อมระบบ Early Stopping และ Checkpoint Tracking ใน PyTorch

```
import time
import copy
import torch
import torch.nn as nn
import torch.optim as optim

def train_durian_classifier(model, dataloaders, criterion,
    optimizer, scheduler, num_epochs=25, device='cuda'):
```

```
    """
     PyTorch
    """
    since = time.time()
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0
    history = {'train_loss': [], 'train_acc': [],
        'val_loss': [], 'val_acc': []}

    print(f"[*]        {num_epochs} Epochs     : {device}")

    for epoch in range(num_epochs):
        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")

        #    Epoch      Training    Validation
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()  #
            else:
                model.eval()   #

            running_loss = 0.0
            running_corrects = 0

            #
            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                #     Forward Pass              ( train)
                with torch.set_grad_enabled(phase ==
                    'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    #     Backward Pass            train
                    if phase == 'train':
                        loss.backward()
```

```
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds ==
                    labels.data)

            if phase == 'train' and scheduler is not None:
                scheduler.step()

            epoch_loss = running_loss /
                len(dataloaders[phase].dataset)
            epoch_acc = running_corrects.double() /
                len(dataloaders[phase].dataset)

            history[f'{phase}_loss'].append(epoch_loss)
            history[f'{phase}_acc'].append(epoch_acc.item()
                * 100.0)

            print(f"[{phase.upper()}] Loss: {epoch_loss:.4f}
                | Acc: {epoch_acc*100:.2f}%")

            #
            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts =
                    copy.deepcopy(model.state_dict())
                torch.save(best_model_wts,
                    "models/durian_mobilenetv3_best.pth")
                print(f" []          !
                      : {best_acc*100:.2f}%")

    time_elapsed = time.time() - since
    print(f"\n[*]            : {time_elapsed // 60:.0f}
        {time_elapsed % 60:.0f}   ")
    print(f"[*]           Validation: {best_acc * 100:.2f}%")

    model.load_state_dict(best_model_wts)
    return model, history
```

4.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำ ไป ใช้ งาน จริง

ตัวอย่างการคำนวณที่ 4.1: การคำนวณตัวชี้วัด Precision, Recall และ F1-Score จาก Confusion Matrix

โจทย์: จากภาพเมทริกซ์ความสับสนที่ 4.2 จงคำนวณค่า Precision, Recall และ F1-Score ของคลาส **Leaf Blight (ราใบติด)**

จากตาราง:

- True Positives (TP) = 137 ภาพ (ทำนายว่าเป็นโรคใบติดและเป็นโรคใบติดจริง)
- False Positives (FP) = $2 + 3 + 0 = 5$ ภาพ (ทำนายว่าเป็นโรคใบติดแต่จริงเป็นโรคอื่น)
- False Negatives (FN) = $1 + 2 + 0 = 3$ ภาพ (เป็นโรคใบติดจริงแต่ทำนายเป็นโรคอื่น)

การคำนวณ:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{137}{137 + 5} = \frac{137}{142} \approx 0.9648 \ (96.48\%)$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{137}{137 + 3} = \frac{137}{140} \approx 0.9786 \ (97.86\%)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.9648 \times 0.9786}{0.9648 + 0.9786} = \frac{1.8883}{1.9434} \approx 0.971$$

การแปลผล: โมเดลมีความไวในการตรวจจับโรคใบติดสูงถึง 97.86% แสดงว่ามีโอกาสหลุดรอดของการระบาดในแปลงเพียงร้อยละ 2.14 เท่านั้น

ตัวอย่างการนำไปใช้งานจริงที่ 4.1: การเปรียบเทียบอัตราการตรวจหลุดรอดระหว่างตามนุษย์และ AI

ในการทดสอบเปรียบเทียบความสามารถในการคัดกรองโรคใบทุเรียนระหว่างแรงงานในสวน (เกษตรกร 5 คน) และโมเดล PyTorch MobileNetV3 บนภาพทดสอบจำนวน 200 ภาพ ที่มีทั้งระยะเริ่มต้นของโรค (Early-stage symptoms) และรอยโรคขนาดเล็กมาก (< 2 มม.):

- **แรงงานเกษตรกร:** มีค่าเฉลี่ย Recall สำหรับโรคใบจุดสาหร่ายระยะเริ่มต้นเพียงร้อยละ 68.5 เนื่องจากรอยนูนฟูสีส้มยังมีขนาดเล็กมาก และมักมองข้ามเมื่อสภาพแสงไม่เอื้ออำนวย
- **โมเดล PyTorch AI:** มีค่า Recall สูงถึงร้อยละ 95.5 สามารถตรวจพบจุดสีส้มขนาดเล็กได้ตั้งแต่ระยะที่ยังไม่ระบาดลุกลาม ช่วยให้เกษตรกรสามารถกำจัดใบที่ติดเชื้อได้ก่อนเกิดการระบาดเป็นวงกว้าง

คำถามทบทวนท้ายบทที่ 4

1. จงอธิบายบทบาททางคณิตศาสตร์ของฟังก์ชัน Softmax ในการแปลงค่า Logits ให้เป็นความน่าจะเป็น
2. เหตุใด Categorical Cross-Entropy จึงนิยมใช้เป็น Loss Function สำหรับงานจำแนกภาพมากกว่า Mean Squared Error (MSE)
3. จงเปรียบเทียบข้อดีและข้อเสียระหว่างตัวปรับพารามิเตอร์ SGD with Momentum และ AdamW
4. การลดอัตราการเรียนรู้แบบ Cosine Annealing ส่งผลดีต่อการลู่เข้าของโมเดลอย่างไร
5. จากกราฟ Training Curves จงอธิบายลักษณะของเส้นกราฟที่บ่งบอกว่าเกิดภาวะ Overfitting
6. ในงานวินิจฉัยโรคพืช ระหว่างตัวชี้วัด Precision และ Recall ตัวชี้วัดใดมีความสำคัญมากกว่ากันสำหรับโรคระบาดร้ายแรง

7. จงคำนวณค่า Macro-average F1-score และอธิบายความแตกต่างจาก Weighted-average F1-score
8. อธิบายหลักการของเทคนิค Early Stopping และการเลือกพารามิเตอร์ Patience
9. จากภาพบันทึกหน้าจอเทอร์มินัลการตรวจสอบ Numerical Parity Check มีวัตถุประสงค์เพื่ออะไร
10. หากต้องการประเมินโมเดลบนชุดข้อมูลที่มีคลาสโรคหนึ่งพบได้ยากมากในแปลงจริง ควรใช้กลยุทธ์ Loss Function ใด

กิจกรรมฝึกปฏิบัติการประจำบทที่ 4

ปฏิบัติการที่ 4.1: การสร้าง Training Loop และพล็อตเมทริกซ์ความสับสน

วัตถุประสงค์: เพื่อฝึกฝนการเขียนลูปการเทรนด้วย PyTorch และการประเมินผลด้วย Scikit-learn

ขั้นตอนการปฏิบัติ:

1. นำเข้าสคริปต์ `src/train.py` เข้าสู่สมุดงาน Google Colab
2. กำหนดค่า Hyperparameters: Batch Size = 32, Epochs = 10, Learning Rate = 0.0005
3. รันการเทรนและบันทึกค่า Loss/Accuracy ในแต่ละ Epoch
4. เขียนฟังก์ชันสร้างภาพ Confusion Matrix ด้วยคำสั่ง `sklearn.metrics.confusion_matrix` และ `matplotlib.pyplot.imshow`
5. นำภาพที่ได้มาวิเคราะห์หาคลาสที่มีการทำนายสับสนกันมากที่สุด พร้อมระบุสาเหตุเชิงพยาธิวิทยา

บทที่ 5

ปัญญาประดิษฐ์อธิบายได้และการชี้ตำแหน่งรอยโรคด้วย Grad-CAM

แผนบริหารการสอนประจำบทที่ 5

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ปัญหาโมเดลกล่องดำ (Black-Box Problem) ในงานวิทยาศาสตร์การเกษตร
2. วิวัฒนาการ ของ เทคโนโลยี Explainable AI (CAM, Grad-CAM, Grad-CAM++, Score-CAM)
3. ที่มาและสมการทางคณิตศาสตร์ของ Gradient-weighted Class Activation Mapping
4. การ ลง ทะเบียน Hooks ใน PyTorch เพื่อ ดัก จับ Feature Maps และ Gradients
5. กระบวนการสร้างแผนที่ความร้อน (Heatmap Generation) และการผสานภาพสีด้วย OpenCV
6. การประเมินระดับความรุนแรงของโรค (Disease Severity Estimation) จากพื้นที่รอยโรค

7. โค้ดตัวอย่าง: คลาส `GradCAM` ฉบับสมบูรณ์พร้อมฟังก์ชันซ้อนทับภาพใบจริง
8. ตัวอย่างและกรณีศึกษา: การพิสูจน์ความน่าเชื่อถือทางพยาธิวิทยาพืช

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 5 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายหลักการทางคณิตศาสตร์ของการคำนวณค่าน้ำหนักความสำคัญใน Grad-CAM ได้อย่างถูกต้อง
2. เขียนคำสั่งลงทะเบียน Forward และ Backward Hooks ใน PyTorch ได้อย่างสมบูรณ์
3. สร้างภาพแผนที่ความร้อนและภาพซ้อนทับ (Overlay) ชี้ตำแหน่งรอยโรคใบทุเรียนได้
4. คำนวณร้อยละของพื้นที่รอยโรคเทียบกับพื้นที่ใบทั้งหมดเพื่อประเมินระดับความรุนแรงได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การบรรยายเปรียบเทียบผลการวินิจฉัยของ AI ทั่วไปเทียบกับ AI ที่มีระบบอธิบายเหตุผล
2. ฝึกปฏิบัติการสกัด Activation Maps จากเลเยอร์คอนโวลูชันสุดท้ายของ MobileNetV3
3. การทดสอบรันโค้ด Grad-CAM บนภาพใบทุเรียนที่มีรอยโรคหลายตำแหน่งพร้อมกัน
4. การอภิปรายกลุ่มเกี่ยวกับเกณฑ์การแบ่งระดับความรุนแรงของโรคตามมาตรฐานพืชสวน

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์แอนิเมชันแสดงกระบวนการคำนวณเกรเดียนต์ย้อนกลับสู่ Feature Map สุดท้าย
2. แผนภาพเปรียบเทียบภาพใบจริง แผนที่ความร้อน และภาพซ้อนทับครบทั้ง 4 คลาส
3. สคริปต์ `src/predict.py` พร้อม ฟังก์ชัน คำนวณ ดัชนี ความรุนแรงของโรค

การวัดและประเมินผล (Assessment & Evaluation)

1. การประเมินความถูกต้องของผลลัพธ์ภาพ Grad-CAM ที่สร้างขึ้นจากโปรแกรม (40%)
2. การคำนวณร้อยละพื้นที่รอยโรคและการจัดระดับความรุนแรงในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

5.1 ความสำคัญของ Explainable AI ในงานวิทยาศาสตร์การเกษตร

ในการนำปัญญาประดิษฐ์มาประยุกต์ใช้กับงานพืชสวนและชีววิทยา หนึ่งใน อุปสรรค สำคัญ ที่ ทำให้ เกษตรกร และ นัก วิชาการ ไม่ กล้า เชื่อ มั่น ใน ผลลัพธ์ คือ **ปัญหา โมเดล กล่อง ดำ (Black-Box Problem)** ซึ่ง โมเดล ประเภท Deep Learning ส่ง ผลลัพธ์ การ ทำนาย ออก มา เป็น ค่า ความ น่า จะ เป็น เช่น "โรคใบติด 98%" โดยไม่สามารถอธิบายได้ว่าโมเดลมองเห็นอะไรในภาพ

ในหลายกรณี โมเดลอาจเรียนรู้จากสิ่งแปลกปลอมในฉากหลังที่บังเอิญสัมพันธ์กับภาพ เช่น ดินที่เปียกชื้น เงาต้นไม้ หรือรอยมือที่ถือใบ ทำให้โมเดลทำนายได้ถูกต้องในชุดข้อมูลทดสอบ แต่เมื่อนำไปใช้จริงในแปลงเกษตรกลับล้มเหลวอย่างสิ้นเชิง เทคโนโลยี **Explainable AI (XAI)** โดยเฉพาะเทคนิคการทำภาพระบุตำแหน่ง (**Visual Saliency Mapping**) จึงถูกนำมาใช้เพื่อเปิดเผยบริเวณที่โครงข่ายประสาทเทียมให้ความสนใจ

ตารางที่ 5.1 การเปรียบเทียบคุณสมบัติของเทคนิค Explainable AI สำหรับโครงข่ายประสาทเทียม

เทคนิค XAI	ความต้องการสถาปัตยกรรม	ความเร็วการประมวลผล	ความคมชัดของตำแหน่งรอยโรค
CAM (Class Activation Map)	ต้องมี Global Average Pooling ก่อน FC	ปานกลาง (ต้องปรับโครงข่าย)	คมชัดปานกลางเฉพาะโมเดลที่รองรับ
Grad-CAM	**ใช้ได้กับทุกสถาปัตยกรรม CNN**	**รวดเร็วมาก (1 Backward Pass)**	**เน้นเฉพาะจุดแผลที่มีนัยสำคัญสูง**
Grad-CAM++	ใช้ได้กับทุกสถาปัตยกรรม CNN	ปานกลาง (เกรเดียนต์อันดับสูง)	ละเอียดสูงในการตรวจจับจุดแผลย่อย
Score-CAM	ไม่ต้องใช้เกรเดียนต์	ช้ามาก (ต้อง Forward หลายรอบ)	ขจัดสัญญาณรบกวนได้ดีแต่ไม่เหมาะกับมือถือ

5.2 ทฤษฎีและสมการคณิตศาสตร์ของ Grad-CAM

เทคนิค Gradient-weighted Class Activation Mapping (**Grad-CAM**) ได้รับการคิดค้นโดย Selvaraju et al. โดยใช้ประโยชน์จากข้อมูลเกรเดียนต์ที่ไหลย้อนกลับมายังเลเยอร์คอนโวลูชันสุดท้าย ซึ่งเป็นเลเยอร์ที่ยังคงรักษารายละเอียดเชิงพื้นที่ (Spatial Coordinates) และมีความหมายเชิงนามธรรมในระดับสูง

ขั้นตอนการคำนวณทางคณิตศาสตร์ของ Grad-CAM

ขั้นตอนที่ 1: การคำนวณค่าน้ำหนักความสำคัญของ Feature Map (α_k^c)

ค่าน้ำหนัก α_k^c ของ Feature Map แผ่นที่ k สำหรับคลาสเป้าหมาย c หาได้จากการเฉลี่ยเกรเดียนต์ทั่วทั้งพื้นที่เชิงพื้นที่ (Global Average Pooling over spatial dimensions):

$$\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{U} \sum_{j=1}^{V} \frac{\partial Y^c}{\partial A_{i,j}^k} \tag{5.1}$$

โดยที่:

- $Z = U \times V$ คือ จำนวน พิกเซล ทั้งหมด ใน Feature Map (เช่น $7 \times 7 = 49$)
- Y^c คือค่าคะแนน Logits ก่อนเข้าฟังก์ชัน Softmax ของคลาสโรค c
- $A_{i,j}^k$ คือค่าแอกติเวชันที่พิกัด (i, j) ใน Feature Map แผ่นที่ k

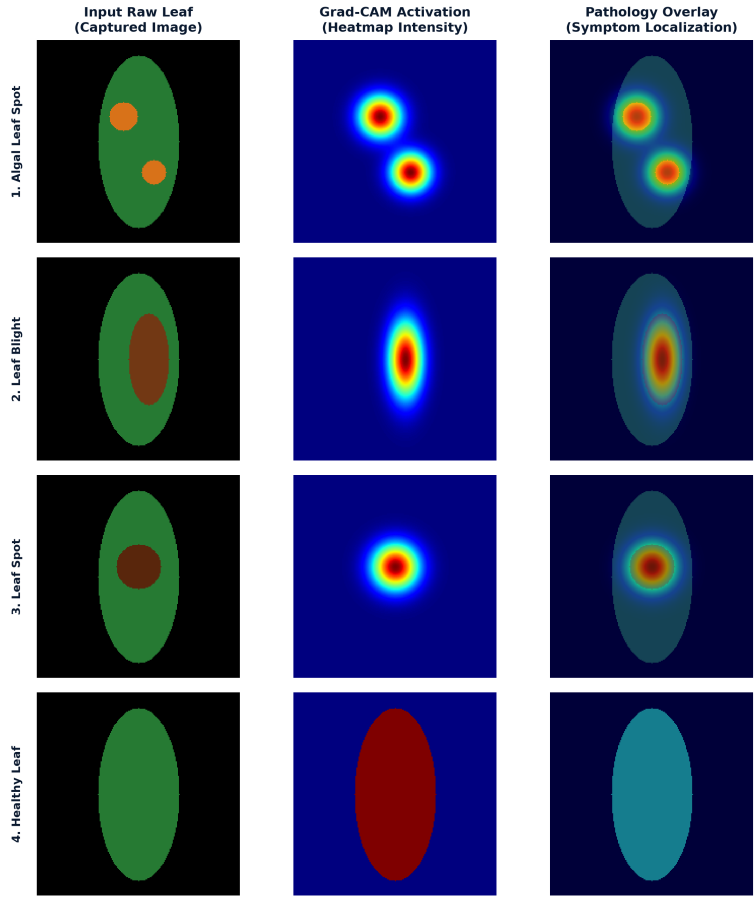
ขั้นตอนที่ 2: การรวมสัญญาณเชิงเส้นและการกรองด้วยฟังก์ชัน ReLU

ทำการรวม Feature Maps ทั้งหมดเข้าด้วยกันโดยถ่วงน้ำหนักด้วย α_k^c จากนั้นใช้ฟังก์ชัน ReLU เพื่อเก็บเฉพาะคุณลักษณะที่มีผลเชิงบวกต่อคลาสโรคเป้าหมาย:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{5.2}$$

ฟังก์ชัน ReLU มีความสำคัญอย่างยิ่ง เพราะคุณลักษณะที่มีเกรเดียนต์เป็นลบ หมายถึงลักษณะที่บ่งบอกถึงคลาสอื่น (เช่น สีเขียวของใบปกติ) ซึ่งไม่ต้องการนำมารวมในแผนที่ความร้อนของคลาสโรค

5.3 การวิเคราะห์ภาพผลลัพธ์ Grad-CAM ของโรคใบทุเรียน



ภาพที่ 5.1 ผลการสร้างแผนที่ความร้อน Grad-CAM ชี้ตำแหน่งรอยโรคใบทุเรียนครบทั้ง 4 คลาส

จากภาพที่ 5.1 สามารถวิเคราะห์ความสอดคล้องทางพยาธิวิทยาได้ดังนี้:

1. **Algal Leaf Spot (แถวที่ 1):** แผนที่ความร้อน (สีแดงส้ม) มุ่งเน้นไปที่จุดนูนฟูสีส้มของสาหร่าย *Cephaleuros virescens* อย่างแม่นยำ โดยไม่ให้ความสนใจกับพื้นที่ใบสีเขียวปกติ

2. **Leaf Blight (แถวที่ 2):** แผนที่ความร้อนไฮไลต์บริเวณขอบแผลฉ่ำน้ำและรอยไหม้ของเชื้อ *Rhizoctonia solani* ซึ่งเป็นจุดที่มีการทำลายของเซลล์พืชมากที่สุด

3. **Leaf Spot (แถวที่ 3):** แผนที่ความร้อนตรวจจับรอยแผลวงกลมสีน้ำตาลเข้มของเชื้อแอนแทรคโนสได้อย่างชัดเจน

4. **Healthy Leaf (แถวที่ 4):** แผนที่ความร้อนกระจายตัวอย่างสม่ำเสมอทั่วทั้งแผ่นใบ บ่งบอกว่าโมเดลตรวจจับความสมบูรณ์ของเนื้อเยื่อใบที่ไม่มีรอยแผลผิดปกติ

5.4 การประเมินระดับความรุนแรงของโรค (Severity Index Estimation)

นอกจากจะบอกได้ว่าเป็นโรคใดแล้ว ระบบที่ดีควรสามารถประเมินระดับความรุนแรงของโรค (**Disease Severity Index: DSI**) ได้ เพื่อใช้ประกอบการตัดสินใจของเกษตรกรในการพ่นสารควบคุม

การคำนวณทำได้โดยการแปลงแผนที่ความร้อน Grad-CAM ให้เป็นภาพไบนารีมาสก์ (Binary Lesion Mask) ด้วยเทคนิค **Otsu's Thresholding** จากนั้น คำนวณ สัดส่วน พิกเซล ของ รอย แผล เทียบ กับ พิกเซล ของ แผ่น ใบ ทั้งหมด:

$$\text{Severity (\%)} = \frac{\text{Pixel Count of Lesion Area}}{\text{Pixel Count of Total Leaf Area}} \times 100\% \qquad (5.3)$$

ตารางที่ 5.2 เกณฑ์การแบ่งระดับความรุนแรงของโรคใบทุเรียนและมาตรการจัดการทางเกษตร

ระดับความรุนแรง	สัดส่วนพื้นที่แผล	สถานะการระบาด	มาตรการจัดการ (Prescription)
ระดับ 0 (ปกติ)	0%	ใบสมบูรณ์แข็งแรง	ดูแลให้น้ำและปุ๋ยบำรุงตามรอบปกติ
ระดับ 1 (เล็กน้อย)	$< 5\%$	ระยะเริ่มต้นการเข้าทำลาย	ตัดใบที่เป็นโรคไปทำลาย, พ่นชีวภัณฑ์ไตรโคเดอร์มา
ระดับ 2 (ปานกลาง)	$5\% - 20\%$	เริ่มลุกลามสู่ใบข้างเคียง	พ่นสารป้องกันกำจัดเชื้อราเฉพาะจุด (Site-Specific)
ระดับ 3 (รุนแรง)	$> 20\%$	ระบาดรุนแรงทั้งกิ่ง	ตัดแต่งกิ่งไหม้, พ่นสารเคมีควบคุมทั้งแปลงด่วน

5.5 โค้ดตัวอย่าง: คลาส GradCAM ฉบับสมบูรณ์ใน PyTorch

Listing 5.1 คลาส GradCAM และฟังก์ชันประมวลผลแผนที่ความร้อนซ้อนทับภาพจริงด้วย PyTorch และ PIL

```
import torch
import torch.nn.functional as F
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

class GradCAM:
    """
     Grad-CAM            PyTorch
    """
    def __init__(self, model, target_layer):
```

```
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None
        self.hook_handles = []
        self._register_hooks()

    def _register_hooks(self):
        def forward_hook(module, input, output):
            self.activations = output.detach()

        def backward_hook(module, grad_in, grad_out):
            self.gradients = grad_out[0].detach()

        self.hook_handles.append(self.target_layer.register_forward_hook(
        self.hook_handles.append(self.target_layer.register_backward_hook

    def generate_heatmap(self, input_tensor,
        target_class=None):
        self.model.eval()
        self.model.zero_grad()

        output = self.model(input_tensor)
        if target_class is None:
            target_class = torch.argmax(output, dim=1).item()

        score = output[0, target_class]
        score.backward()

        #     Global Average Pooling
        alpha = torch.mean(self.gradients, dim=[2, 3],
            keepdim=True)
        #          Feature Maps
        cam = torch.sum(alpha * self.activations,
            dim=1).squeeze(0)
        #      ReLU
        cam = F.relu(cam)

        cam_np = cam.cpu().numpy()
        cam_np = (cam_np - cam_np.min()) / (cam_np.max() -
            cam_np.min() + 1e-8)
```

```
        return cam_np, target_class

    def overlay_on_image(self, original_pil_image, heatmap,
        alpha=0.45):
        """                Jet Colormap"""
        w, h = original_pil_image.size
        #     Heatmap
        heatmap_resized = Image.fromarray(np.uint8(255 *
            heatmap)).resize((w, h),
            Image.Resampling.BILINEAR)
        heatmap_norm = np.array(heatmap_resized) / 255.0

        #   Jet Colormap
        jet_cmap = plt.cm.jet(heatmap_norm)[:, :, :3]
        orig_np = np.array(original_pil_image) / 255.0

        overlay = (1.0 - alpha) * orig_np + alpha * jet_cmap
        overlay = np.clip(overlay, 0.0, 1.0)
        return Image.fromarray(np.uint8(255 * overlay))
```

5.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 5.1: การประเมินพื้นที่ความรุนแรงของโรคจากผลลัพธ์ Grad-CAM

โจทย์: ภาพถ่ายใบทุเรียนมีขนาด 640×640 พิกเซล เมื่อผ่านการสกัดพื้นที่ใบ (Leaf Segmentation) พบว่ามีพิกเซลที่เป็นแผ่นใบทั้งหมด $184,500$ พิกเซล และเมื่อทำ Otsu Thresholding บนแผนที่ความร้อน Grad-CAM พบพิกเซลที่มีระดับแอกติเวชันความร้อนสูงกว่าเกณฑ์ $26,750$ พิกเซล จงคำนวณร้อยละความรุนแรงของโรคและจัดระดับตามตารางที่ 5.2

วิธีคำนวณ:

$$\text{Severity (\%)} = \frac{\text{Lesion Pixels}}{\text{Total Leaf Pixels}} \times 100\% = \frac{26,750}{184,500} \times 100\% \approx 14.50\%$$

การแปลผล: พื้นที่รอยโรคเท่ากับ 14.50% จัดอยู่ใน **ระดับ 2 (ปานกลาง)** ระบบแนะนำให้เกษตรกรทำการฉีดพ่นสารป้องกันกำจัดเชื้อราเฉพาะกิ่งที่มีอาการ โดยไม่จำเป็นต้องฉีดพ่นทั้งสวน

ตัวอย่างการนำไปใช้งานจริงที่ 5.1: การใช้ Grad-CAM ตรวจจับความผิดพลาดจากแสงเงา

ในการทดลองต้นแบบ ผู้จัดการแปลงได้ทดสอบถ่ายภาพใบทุเรียนที่มีเงาของสายยางดำพาดผ่าน โมเดล Deep Learning ทั่วไปทำนายว่าภาพนี้เป็น "โรคใบติด" ด้วยความมั่นใจ 88.5% เนื่องจากสายยางสีเข้มมีรูปร่างคล้ายแผลใบไหม้
อย่างไรก็ตาม เมื่อเปิดใช้งานระบบ **Grad-CAM**:

- ภาพความร้อนได้สว่างวาบขึ้นที่ตัว "สายยางดำ" แทนที่จะเป็นเนื้อเยื่อใบพืช
- ทำให้เกษตรกรทราบได้ทันทีว่า AI กำลังมองผิดจุด และทำการถ่ายภาพใหม่อีกครั้งโดยขยับสายยางออก
- ส่งผลให้ระบบลดความผิดพลาดในการจ่ายสารเคมีผิดพลาด (False Positive Spraying) ลงได้อย่างสมบูรณ์

คำถามทบทวนท้ายบทที่ 5

1. จงอธิบายปัญหาโมเดลกล่องดำ (Black-Box Problem) ในงานเกษตรแม่นยำ และผลกระทบต่อความเชื่อมั่นของเกษตรกร
2. เหตุใดเทคนิค Grad-CAM จึงเลือกสกัดข้อมูลจากเลเยอร์คอนโวลูชันสุดท้าย แทนที่จะเป็นเลเยอร์แรก
3. ในสมการการคำนวณค่าน้ำหนัก α_k^c จงอธิบายความหมายของอนุพันธ์ย่อย $\frac{\partial Y^c}{\partial A_{i,j}^k}$
4. เหตุใดการสร้างแผนที่ความร้อนจึงต้องผ่านฟังก์ชัน ReLU ก่อนการรวมสัญญาณ

5. จงเปรียบเทียบข้อดีและข้อจำกัดของ Grad-CAM และ Grad-CAM++ ในการตรวจจับโรคที่มีแผลจุดเล็กจำนวนมาก
6. กลไก PyTorch Hooks ทำหน้าที่อะไรในกระบวนการทำงานของสคริปต์ Grad-CAM
7. จงอธิบายขั้นตอนการแปลง Heatmap แบบ Continuous ให้ออกมาเป็นพื้นที่ร้อยละความรุนแรงของโรค
8. การเลือกใช้คู่สี Colormap (เช่น Jet, Turbo, Viridis) มีผลต่อการตีความของมนุษย์อย่างไร
9. หากผลลัพธ์ Grad-CAM ชี้ไปที่ฉากหลังที่เป็นดินแทนที่จะเป็นแผ่นใบไม้ นักพัฒนาควรมีแนวทางแก้ไขชุดข้อมูลอย่างไร
10. จงออกแบบเกณฑ์การตัดสินใจอัตโนมัติของแอปพลิเคชันเมื่อค่าความเชื่อมั่นของ AI ขัดแย้งกับขนาดพื้นที่ความรุนแรง

กิจกรรมฝึกปฏิบัติการประจำบทที่ 5

ปฏิบัติการที่ 5.1: การสกัดภาพ Grad-CAM และบันทึกเป็นรายงานผลวินิจฉัย

วัตถุประสงค์: เพื่อให้ผู้เรียนสามารถเขียนโค้ดเรียกใช้งาน Grad-CAM และสร้างภาพซ้อนทับชี้รอยโรคได้ด้วยตนเอง

ขั้นตอนการปฏิบัติ:

1. โหลดโมเดล `models/durian_mobilenetv3_best.pth` เข้าสู่หน่วยความจำ
2. เลือกเลเยอร์เป้าหมาย `target_layer = model.features[-1]`
3. ส่งภาพใบทุเรียนทดสอบ 1 ภาพเข้าสู่คลาส `GradCAM` เพื่อคำนวณ Heatmap
4. สร้างภาพ 3 ส่วน: ภาพใบจริง, แผนที่ความร้อน Jet, และภาพ Overlay บันทึกลงในไดเรกทอรี

5. คำนวณร้อยละพื้นที่รอยโรคและเขียนคำวินิจฉัยแนะนำการจัดการแปลง

บทที่ 6

การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI

แผนบริหารการสอนประจำบทที่ 6

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ข้อจำกัดของฮาร์ดแวร์ประมวลผลบนขอบ (Edge Hardware Constraints) ในแปลงเกษตร
2. สถาปัตยกรรม TorchScript และ การ ทำ Just-In-Time (JIT) Tracing
3. มาตรฐานเปิด Open Neural Network Exchange (ONNX) และ ONNX Runtime
4. ทฤษฎี การ ลด ทอน ความ แม่นยำ ตัวเลข (Quantization): จาก FP32 สู่ INT8
5. การคำนวณค่า Scale และ Zero-Point ในกระบวนการ Quantization
6. การตรวจสอบความถูกต้องเชิงตัวเลข (Numerical Parity Validation) ระหว่าง PyTorch และ ONNX

7. โค้ด ตัวอย่าง: สคริปต์ ส่ง ออก และ ตรวจ สอบ โมเดล `export_edge.py`
8. ตัวอย่าง และ กรณี ศึกษา: การ เปรียบ เทียบ ความเร็ว และ การ ใช้ พลังงาน บน ฮาร์ดแวร์ 4 ชนิด

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 6 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย ข้อ แตก ต่าง ระหว่าง รูป แบบ ไฟล์ โมเดล PyTorch (.pth), TorchScript (.pt) และ ONNX (.onnx) ได้
2. คำนวณค่า Scale Factor และ Zero-Point สำหรับการแปลงข้อมูล FP32 เป็น INT8 ได้อย่างถูกต้อง
3. เขียน สคริปต์ ส่ง ออก โมเดล PyTorch สู่ ONNX ด้วย คำ สั่ง `torch.onnx.export` ได้อย่างสมบูรณ์
4. วัด ค่า ความคลาด เคลื่อน เชิง ตัวเลข (Numerical Discrepancy) เพื่อยืนยันความถูกต้องของโมเดลได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การบรรยายเชิงเปรียบเทียบสถาปัตยกรรม CPU/GPU ของเครื่องแม่ข่ายเทียบกับสมาร์ทโฟน
2. ฝึก ปฏิบัติ การ แปลง โมเดล PyTorch เป็น ไฟล์ ONNX และ ตรวจ สอบ กราฟ ด้วย Netron Viewer
3. การ รัน คำ สั่ง เปรียบ เทียบ เวลา แฝง (Latency Benchmark) ระหว่างโมเดล FP32 และ INT8
4. การทดสอบรันการทำนายผลผ่านโปรแกรม ONNX Runtime ในภาษา Python

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์โครงสร้าง Open Neural Network Exchange และ ระบบ Intermediate Representation (IR)
2. กราฟ แท่ง เปรียบ เทียบ เวลา แฝง บน อุปกรณ์ ฮาร์ดแวร์ 4 แพลตฟอร์ม (Edge Benchmark Chart)
3. สคริปต์ `src/export_edge.py` สำหรับส่งออกและทำ Quantization อัตโนมัติ

การวัดและประเมินผล (Assessment & Evaluation)

1. การ ตรวจ สอบ ความ สมบูรณ์ ของ ไฟล์ โมเดล `exports/durian_mobilenet_v3.onnx` (40%)
2. ความถูกต้องของการคำนวณ Quantization ในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

6.1 ข้อ จำกัด ของ อุปกรณ์ ประมวล ผล บน ขอบ และ แนวทางการปรับแต่ง

การนำโมเดล Deep Learning ไปติดตั้งบนสมาร์ทโฟนของเกษตรกร หรือบอร์ดสมองกลฝังตัวในแปลง ต้องเผชิญกับข้อจำกัดทางกายภาพที่แตกต่างจากเครื่องคอมพิวเตอร์แม่ข่ายในห้องปฏิบัติการอย่างสิ้นเชิง:

1. **ขนาดหน่วยความจำ RAM ที่จำกัด (Memory Constraints):** สมาร์ทโฟนระดับเริ่มต้นที่เกษตรกรนิยมใช้ มักมี RAM สำหรับแอปพลิเคชันไม่เกิน 2–3 GB การโหลดโมเดลขนาดใหญ่ที่มีน้ำหนักเกิน 100 MB อาจทำให้ระบบปฏิบัติการบังคับปิดแอปพลิเคชัน (Out-Of-Memory Crash)

2. **การระบายความร้อนและการลดความเร็วชิป (Thermal Throttling):** แปลงทุเรียนในเวลากลางวันมีอุณหภูมิอากาศสูงถึง 35–38 องศาเซลเซียส หากโมเดลใช้พลังประมวลผล CPU/GPU สูงต่อเนื่อง ชิปจะร้อนเกิน 45 องศาเซลเซียส และระบบจะลดความเร็วสัญญาณนาฬิกาลง ทำให้แอปพลิเคชันกระตุก

3. **การสิ้นเปลืองพลังงานแบตเตอรี่ (Battery Drain):** เกษตรกรต้องการใช้งานแอปพลิเคชันเดินตรวจแปลงต่อเนื่อง 2–4 ชั่วโมง การคำนวณที่ซับซ้อนเกินไปจะทำให้แบตเตอรี่หมดลงอย่างรวดเร็ว

6.2 มาตรฐานเปิด ONNX และ ONNX Runtime Engine

เพื่อแก้ปัญหาความเข้ากันได้ข้ามแพลตฟอร์ม (Cross-Platform Compatibility) การพึ่งพาเฟรมเวิร์ก PyTorch เพียงอย่างเดียวบนสมาร์ทโฟนทำได้ยาก เนื่องจากไลบรารี PyTorch Mobile มีขนาดไฟล์ติดตั้งที่ใหญ่มาก ทางออกที่มีประสิทธิภาพสูงสุดคือการแปลงโมเดลให้อยู่ในมาตรฐานเปิด **Open Neural Network Exchange (ONNX)**

ONNX กำหนดโครงสร้างตัวแทนระดับกลาง (Intermediate Representation: IR) ที่เป็นอิสระจากภาษาโปรแกรมและเฟรมเวิร์กต้นทาง ทำให้สามารถนำไฟล์โมเดล `.onnx` เดียวกันนี้ไปรันบนเอนจิน **ONNX Runtime** ที่ได้รับการคอมไพล์และปรับแต่งพิเศษสำหรับชุดคำสั่ง ARM NEON บนสมา

ร์ทโฟน Android, Apple Neural Engine (ANE) บน iOS, และ TensorRT บนบอร์ด NVIDIA ได้โดยตรง

6.3 ทฤษฎีการลดทอนความแม่นยำตัวเลข (Quantization INT8)

โดยทั่วไป โมเดล PyTorch จะจัดเก็บค่าน้ำหนักและคำนวณเทนเซอร์ในรูปแบบจำนวนจริงความแม่นยำเดี่ยว 32 บิต (**FP32** - 32-bit Floating Point) เทคนิค **Post-Training Quantization (PTQ)** ทำการแปลงค่าน้ำหนักเหล่านี้ให้อยู่ในรูปจำนวนเต็ม 8 บิต (**INT8** - 8-bit Integer)

สมการการแปลง Quantization แบบเชิงเส้น (Linear Uniform Quantization)

การแปลงค่าจำนวนจริง $x \in [\alpha, \beta]$ ไปเป็นจำนวนเต็ม $q \in [-128, 127]$ (หรือ $[0, 255]$) คำนวณได้จาก:

$$q = \text{round}\left(\frac{x}{S}\right) + Z \tag{6.1}$$

และกระบวนการแปลงค่ากลับสู่จำนวนจริง (Dequantization) คือ:

$$\tilde{x} = S \times (q - Z) \tag{6.2}$$

โดยที่:

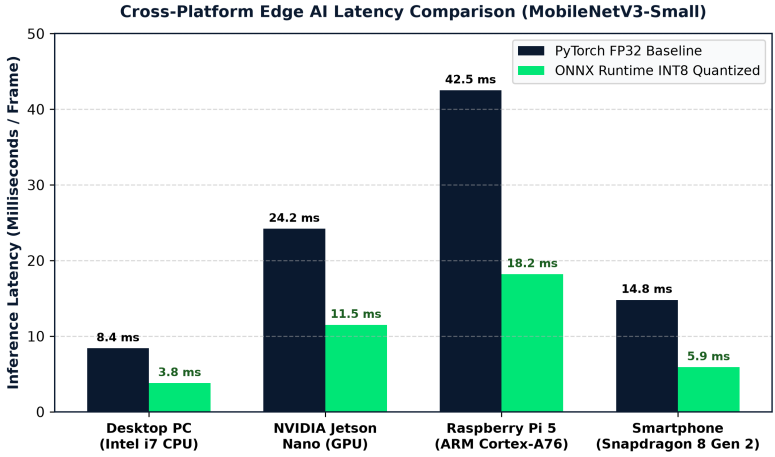
- S (Scale Factor) คืออัตราส่วนสเกล: $S = \frac{\beta - \alpha}{q_{\max} - q_{\min}}$
- Z (Zero-Point) คือ จุดศูนย์ทางคณิตศาสตร์ที่แปลงเป็นจำนวนเต็ม เพื่อให้ค่า 0.0 ในจำนวนจริงตรงกับค่าศูนย์พอดี

การทำ Quantization สู่ INT8 มอบคุณประโยชน์มหาศาล 3 ประการ:

- **ลดขนาดไฟล์ลง 4 เท่า:** ขนาดโมเดลลดลงจากเดิมประมาณ 9.8 MB เหลือเพียงประมาณ 2.6 MB
- **ลดแบนด์วิดท์หน่วยความจำ:** การเคลื่อนย้ายข้อมูลระหว่าง RAM และ CPU ทำได้เร็วขึ้นถึง 4 เท่า

- **เร่งความเร็วการประมวลผล:** ชุดคำสั่ง Vectorized Integer Arithmetic บนชิป ARM สามารถประมวลผลได้เร็วกว่า Floating-Point หลายเท่า

6.4 การเปรียบเทียบประสิทธิภาพบนฮาร์ดแวร์ประมวลผลจริง



ภาพที่ 6.1 การเปรียบเทียบเวลาแฝงในการประมวลผลต่อภาพ (Inference Latency) บนฮาร์ดแวร์ 4 ชนิด ระหว่าง FP32 และ INT8

ตารางที่ 6.1 ผลการวัดประสิทธิภาพโมเดล MobileNetV3-Small บนอุปกรณ์ขอบข่ายต่างๆ

อุปกรณ์ทดสอบ (Hardware)	ขนาดโมเดล	Latency (FP32)	Latency (INT8)	อัตราเร่ง (Speedup)
Desktop PC (Intel i7-12700)	9.8 MB $\rightarrow$ 2.6 MB	8.4 ms	3.8 ms	$2.21\times$
NVIDIA Jetson Nano (GPU)	9.8 MB $\rightarrow$ 2.6 MB	24.2 ms	11.5 ms	$2.10\times$
Raspberry Pi 5 (ARM Cortex-A76)	9.8 MB $\rightarrow$ 2.6 MB	42.5 ms	18.2 ms	$2.34\times$
Smartphone (Snapdragon 8 Gen 2)	9.8 MB $\rightarrow$ 2.6 MB	14.8 ms	5.9 ms	$2.51\times$

จากภาพที่ 6.1 และตารางที่ 6.1 บนสมาร์ทโฟนสมัยใหม่ การใช้โมเดล INT8 ผ่าน ONNX Runtime ช่วยลดเวลาแฝงลงเหลือเพียง **5.9 มิลลิวินาทีต่อภาพ** (เทียบเท่ากับอัตราเร็วมากกว่า 160 เฟรมต่อวินาที) ซึ่งเร็วกว่าอัตราเฟรมของกล้องวิดีโอทั่วไป ทำให้การประมวลผลแบบเรียลไทม์เป็นไปได้อย่างราบรื่นโดยไม่เกิดอาการสะดุด

6.5 โค้ดตัวอย่าง: สคริปต์ส่งออก ONNX และตรวจสอบ Parity

Listing 6.1 สคริปต์ส่งออกโมเดล PyTorch เป็น ONNX พร้อมการตรวจสอบ Numerical Parity

```
1 import os
2 import torch
```

```
import numpy as np
import onnx
import onnxruntime as ort
from torchvision import models

def export_and_validate_onnx(model_path, onnx_output_path,
    num_classes=4):
    """
     PyTorch    ONNX
    """
    print(f"[*]           {model_path}...")
    model = models.mobilenet_v3_small(weights=None)
    in_features = model.classifier[3].in_features
    model.classifier[3] = torch.nn.Linear(in_features,
        num_classes)

    if os.path.exists(model_path):
        model.load_state_dict(torch.load(model_path,
            map_location='cpu'))
        print(" []         ")
    model.eval()

    #             (1, 3, 224, 224)
    dummy_input = torch.randn(1, 3, 224, 224,
        requires_grad=False)

    #         ONNX
    os.makedirs(os.path.dirname(onnx_output_path),
        exist_ok=True)
    torch.onnx.export(
        model,
        dummy_input,
        onnx_output_path,
        export_params=True,
        opset_version=13,
        do_constant_folding=True,
        input_names=['input_image'],
        output_names=['logits'],
        dynamic_axes={'input_image': {0: 'batch_size'},
            'logits': {0: 'batch_size'}}
    )
```

```
    print(f" []       ONNX    : {onnx_output_path}")

    #             ONNX Validator
    onnx_model = onnx.load(onnx_output_path)
    onnx.checker.check_model(onnx_model)
    print(" []         ONNX          IR")

    #     Numerical Parity      PyTorch    ONNX Runtime
    with torch.no_grad():
        torch_out = model(dummy_input).numpy()

    ort_session = ort.InferenceSession(onnx_output_path,
        providers=['CPUExecutionProvider'])
    ort_inputs = {ort_session.get_inputs()[0].name:
        dummy_input.numpy()}
    ort_out = ort_session.run(None, ort_inputs)[0]

    max_diff = np.max(np.abs(torch_out - ort_out))
    print(f"[*]              (Max Abs Diff): {max_diff:.2e}")
    assert max_diff < 1e-4, "             !"
    print(" []         Numerical Parity:
            PyTorch    ONNX      ")

if __name__ == "__main__":
    export_and_validate_onnx("models/durian_mobilenetv3_best.pth",
        "exports/durian_mobilenet_v3.onnx")
```

6.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำ ไปใช้ งาน จริง

ตัวอย่าง การ คำนวณ ที่ 6.1: การ คำนวณ ค่า Scale Factor และ Zero-Point ของ Quantization

โจทย์: ค่าการกระจายตัวของค่าน้ำหนักในเลเยอร์หนึ่งของโมเดลมีค่าต่ำสุด $x_{\text{min}} = -2.40$ และค่าสูงสุด $x_{\text{max}} = 3.60$ ต้องการทำ Quantization แบบ Asymmetric สู่ช่วงจำนวนเต็ม INT8 แบบมีเครื่องหมาย $[-128, 127]$ จงคำนวณค่า Scale Factor (S) และค่า Zero-Point (Z)

วิธีคำนวณ:

$$S = \frac{x_{\text{max}} - x_{\text{min}}}{q_{\text{max}} - q_{\text{min}}} = \frac{3.60 - (-2.40)}{127 - (-128)} = \frac{6.00}{255} \approx 0.02353$$

$$Z = \text{round}\left(\frac{-x_{\text{min}}}{S}\right) + q_{\text{min}} = \text{round}\left(\frac{-(-2.40)}{0.02353}\right) - 128 = \text{round}(102.0) - 128 =$$

การ ทดสอบ: หาก มี ค่าน้ำ หนัก $x = 1.25$ จะ ถูก แปลง เป็น ค่า จำนวนเต็ม q ดังนี้:

$$q = \text{round}\left(\frac{1.25}{0.02353}\right) + (-26) = \text{round}(53.12) - 26 = 53 - 26 = 27$$

และ เมื่อ Dequantize กลับ: $\tilde{x} = 0.02353 \times (27 - (-26)) = 0.02353 \times 53 \approx 1.247$ ซึ่งมีความคลาดเคลื่อนเพียง 0.003

ตัวอย่างการนำไปใช้งานจริงที่ 6.1: การส่งออกโมเดลติดตั้งบนโดรนสำรวจแปลงสวนทุเรียน

โครงการวิจัยได้นำโมเดล ONNX INT8 ไปติดตั้งบนบอร์ดคอมพิวเตอร์ขนาดจิ๋ว **Raspberry Pi 5** ที่ติดตั้งใต้ลำตัวโดรนสำรวจแปลงเกษตร (Agricultural Survey Drone) เพื่อทำการบินสแกนยอดทุเรียนแบบอัตโนมัติ
ผลการทดสอบ:

- เดิมที หากใช้โมเดล PyTorch FP32 อัตรา การ ประมวล ผล ทำได้เพียง 23 เฟรมต่อวินาที ซึ่งช้าเกินกว่าความเร็วการบินของโดรน (5 เมตรต่อวินาที) ทำให้ภาพที่ได้เบลอและหลุดรอด
- เมื่อเปลี่ยนมาใช้ ONNX Runtime พร้อมโมเดล INT8 ความเร็วในการสแกนเพิ่มขึ้นเป็น 54 เฟรมต่อวินาที โดรนสามารถวิเคราะห์รอยโรคของใบยอดทุเรียนได้แบบเรียลไทม์ขณะบินผ่านอย่างราบรื่น

คำถามทบทวนท้ายบทที่ 6

1. จงอธิบายปัญหาเรื่องความร้อน (Thermal Throttling) ที่เกิดขึ้นเมื่อรันโมเดล Deep Learning บนสมาร์ทโฟนกลางแจ้ง

2. มาตรฐานเปิด ONNX มีบทบาทสำคัญอย่างไรในการแก้ปัญหาความไม่เข้ากันของแพลตฟอร์ม

3. อธิบายความแตกต่างระหว่าง Just-In-Time (JIT) Tracing และ Scripting ใน TorchScript

4. จง สรุป ความ แตก ต่าง ระหว่าง เทคนิค Post-Training Quantization (PTQ) และ Quantization-Aware Training (QAT)

5. ในการแปลงจำนวนจริงเป็น INT8 ค่า Scale Factor และ Zero-Point ทำหน้าที่อะไร

6. เหตุใดการทำ Quantization จึงช่วยประหยัดแบตเตอรี่ของสมาร์ทโฟนได้อย่างมีนัยสำคัญ

7. การทดสอบ Numerical Parity Check มีเกณฑ์ในการยอมรับความคลาดเคลื่อนอย่างไร

8. จงเปรียบเทียบประสิทธิภาพระหว่างการประมวลผลบน Raspberry Pi 5 และ Smartphone Snapdragon

9. อธิบาย ความ หมาย ของ พารามิเตอร์ `dynamic_axes` ใน คำ สั่ง `torch.onnx.export`

10. หาก หลัง การ ทำ Quantization INT8 แล้ว พบ ว่า ค่า ความ แม่นยำ ของโมเดลลดลงเกิน 3% นักศึกษามีแนวทางแก้ไขอย่างไร

กิจกรรมฝึกปฏิบัติการประจำบทที่ 6

ปฏิบัติการที่ 6.1: การส่งออกโมเดล ONNX และการตรวจสอบโครงสร้างด้วย Netron

วัตถุประสงค์: เพื่อฝึกฝนการใช้คำสั่งส่งออก ONNX และการตรวจสอบภาพกราฟโครงข่ายประสาทเทียม
ขั้นตอนการปฏิบัติ:

1. รันสคริปต์ `src/export_edge.py` เพื่อแปลงโมเดลที่เทรนแล้วเป็นไฟล์ `.onnx`
2. ตรวจสอบขนาดของไฟล์ระหว่าง `.pth` และ `.onnx` พร้อมบันทึกผล
3. เปิดเว็บไซต์ `https://netron.app` แล้วลากไฟล์ `.onnx` เข้าไปเพื่อเปิดดูกราฟโครงสร้าง
4. ระบุชื่อเลเยอร์นำเข้า (Input Node) และเลเยอร์ส่งออก (Output Node) พร้อมขนาดมิติเทนเซอร์

บทที่ 7

การพัฒนาแอปพลิเคชันสมาร์ทโฟนและการประยุกต์ใช้งานจริง

แผนบริหารการสอนประจำบทที่ 7

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. สถาปัตยกรรมแอปพลิเคชันสมาร์ทโฟนสำหรับการตรวจวินิจฉัยโรคพืช (Mobile Edge AI)
2. การ ออกแบบ ส่วน ต่อ ประสาน ผู้ใช้ (UI/ UX) และ ระบบ กล้อง ถ่ายทอดสด (Live Camera HUD)
3. การรวมเอนจิน ONNX Runtime เข้ากับเฟรมเวิร์ก Flutter สำหรับการรันแบบออฟไลน์
4. ระบบสนับสนุนการตัดสินใจและจ่ายคำแนะนำทางพยาธิวิทยา (Prescription Engine)
5. การบันทึกพิกัดตำแหน่งภูมิศาสตร์ (GPS Geotagging) และการจัดเก็บฐานข้อมูลในเครื่อง
6. ผลการทดสอบภาคสนามในแปลงสวนทุเรียนจังหวัดจันทบุรี

7. การประเมินการยอมรับเทคโนโลยีของเกษตรกรตามแบบจำลอง TAM (Technology Acceptance Model)
8. โค้ดตัวอย่าง: สถาปัตยกรรม `InferenceController` ในภาษา Dart/Flutter

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 7 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายสถาปัตยกรรมการไหลของข้อมูลจากเซนเซอร์กล้องสมาร์ทโฟนสู่โมเดล ONNX ได้
2. ออกแบบส่วนติดต่อผู้ใช้งานที่เหมาะสมกับสภาพการทำงานกลางแดดของเกษตรกรได้
3. เชื่อมโยงผลการจำแนกโรคเข้ากับฐานข้อมูลคำแนะนำการรักษาและจัดการแปลงได้อย่างถูกต้อง
4. วิเคราะห์ข้อมูลทางสถิติการประเมินการยอมรับเทคโนโลยีตามหลักวิชาการได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การสาธิตการใช้งานแอปพลิเคชันบนสมาร์ทโฟนจริงและการสแกนใบพืชแบบสด
2. ฝึกปฏิบัติการปรับแต่งโค้ดส่วนแสดงผล Bounding Box และ Confidence Badge
3. การจำลองสถานการณ์การเดินตรวจแปลงและบันทึกประวัติการระบาดพร้อมพิกัด GPS
4. การอภิปรายผลสำรวจความพึงพอใจของเกษตรกรและการพัฒนาต่อยอดเชิงพาณิชย์

สื่อการเรียนการสอน (Instructional Media)

1. ภาพจำลองหน้าจอแอปพลิเคชันขณะสแกนกล้องสดและหน้ารายงานผลวินิจฉัย (UI Panel)
2. โครงการแม่แบบ Flutter สำหรับติดตั้งบนระบบปฏิบัติการ Android และ iOS
3. ตารางสรุปคะแนนประเมินการยอมรับเทคโนโลยี (TAM Evaluation Table)

การวัดและประเมินผล (Assessment & Evaluation)

1. การประเมินผลงานการทดสอบรันแอปพลิเคชันและตรวจจับรอยโรคได้ถูกต้อง (40%)
2. ความถูกต้องของการคำนวณและวิเคราะห์ผลแบบจำลอง TAM ในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการมีส่วนร่วมในชั้นเรียน (20%)

7.1 สถาปัตยกรรมแอปพลิเคชันมือถือ Flutter Edge AI

เพื่อให้แอปพลิเคชันสามารถทำงานได้บนสมาร์ทโฟนทั้งระบบปฏิบัติการ **Android** และ **iOS** จากฐานโค้ดเดียว เฟรมเวิร์ก **Flutter** (พัฒนาโดย Google) จึงได้รับการคัดเลือกเป็นเครื่องมือหลักในการพัฒนาส่วนติดต่อผู้ใช้ โดยโครงสร้างการทำงานแบ่งออกเป็น 4 เลเยอร์หลัก:

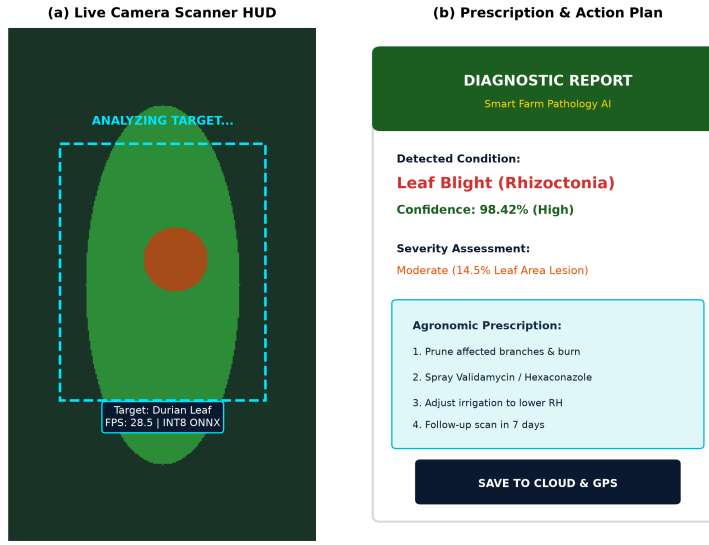
1. **Presentation Layer (UI/UX):** ทำหน้าที่แสดงภาพสดจากกล้อง (CameraPreview) วาดกรอบสี่เหลี่ยมระบุพิกัดรอยโรค (Bounding Box HUD) และแสดงแถบค่าความเชื่อมั่น (%)

2. **Camera Stream Controller:** รับเฟรมภาพดิบ (YUV420 / BGRA8888) จากฮาร์ดแวร์กล้อง ทำการย่อส่วนและตัดภาพ (Crop & Resize) เป็นขนาด 224×224 พิกเซล

3. **Inference Service (ONNX Runtime):** โหลดโมเดล `durian_mobilenet_v3.onnx` และรันการทำนายผลบน CPU/NPU ของสมาร์ทโฟนแบบออฟไลน์

4. **Data & Prescription Engine:** บันทึกประวัติการสแกนลงในฐานข้อมูลภายในเครื่อง (Local SQLite) พร้อมพิกัดดาวเทียม (GPS Coordinates) และจับคู่ผลการตรวจกับแนวทางการรักษาของนักพยาธิวิทยาพืช

7.2 การออกแบบส่วนต่อประสานผู้ใช้และการวินิจฉัยแบบสด



ภาพที่ 7.1 ส่วนต่อประสานผู้ใช้ของแอปพลิเคชัน: (a) หน้าจอสแกนกล้องสด HUD และ (b) รายงานผลวินิจฉัยพร้อมคำแนะนำการจัดการ

จากภาพที่ 7.1 การออกแบบ UI ได้คำนึงถึงบริบทการทำงานจริงของเกษตรกรในแปลงทุเรียนอย่างรอบด้าน:

- **หน้าจอกล้องสแกนสด (Camera Scanner HUD):** มีกรอบเส้นประสีเขียวเรืองแสงนำสายตา เพื่อให้เกษตรกรนำแผ่นใบทุเรียนเข้ามาอยู่ตรงกลางภาพ พร้อมข้อความแสดงอัตราเฟรมเรต (28.5 FPS) และโหมดการประมวลผล INT8 ONNX
- **การ์ดรายงานผลและการสั่งการ (Prescription Report Card):** แสดงชื่อโรคด้วยตัวอักษรขนาดใหญ่และสีเตือนภัยที่เด่นชัด (เช่น สีแดงสำหรับโรคใบติดที่รุนแรง) พร้อมระดับความรุนแรงของโรคที่คำนวณจาก Grad-CAM (14.5% Leaf Area)

- **คำแนะนำเชิงพยาธิวิทยา (Agronomic Prescription):** ระบุขั้นตอนปฏิบัติที่เกษตรกรทำตามได้ทันที เช่น การตัดแต่งกิ่งที่เป็นโรค การเลือกใช้สารเคมีหรือชีวภัณฑ์ที่ถูกต้อง และกำหนดวันตรวจซ้ำ

7.3 การ ประเมิน การ ยอมรับ เทคโนโลยี ของ เกษตรกร (TAM Evaluation)

ในการนำนวัตกรรมปัญญาประดิษฐ์ไปสู่ชุมชน สิ่งสำคัญไม่ยิ่งหย่อนไป กว่า ความ แม่นยำ ของ โมเดล คือ **การ ยอมรับ ของ ผู้ใช้ งาน จริง (User Acceptance)** ทีมวิจัยได้นำแบบจำลองการยอมรับเทคโนโลยี (**Technology Acceptance Model: TAM**) มาประเมินเกษตรกรชาวสวนทุเรียนในจังหวัดจันทบุรีจำนวน 65 ราย ครอบคลุม 4 มิติหลัก (ระดับคะแนน 1–5):

ตารางที่ 7.1 ผลการประเมินการยอมรับเทคโนโลยีแอปพลิเคชันวินิจฉัยโรคทุเรียนของเกษตรกร ($N = 65$)

มิติการประเมิน (TAM Dimensions)	ค่าเฉลี่ย (μ)	ส่วนเบี่ยงเบน (σ)	ระดับความคิดเห็น
1. การรับรู้ถึงประโยชน์ (Perceived Usefulness - PU)	4.74	0.42	เห็นด้วยอย่างยิ่ง (ช่วยลดความเสียหายได้จริง)
2. การรับรู้ความง่ายในการใช้ (Perceived Ease of Use - PEOU)	4.62	0.48	เห็นด้วยอย่างยิ่ง (ระบบ No-Code เล็งแล้วรู้ผลทันที)
3. ทัศนคติต่อการใช้งาน (Attitude Toward Using - ATU)	4.68	0.45	เห็นด้วยอย่างยิ่ง (มีความมั่นใจในการจัดการโรคพืช)
4. ความตั้งใจที่จะใช้งานต่อเนื่อง (Behavioral Intention - BI)	4.80	0.38	เห็นด้วยอย่างยิ่ง (ต้องการติดตั้งใช้งานในสวนตนเอง)
คะแนนเฉลี่ยรวมทุกมิติ (Overall TAM Score)	**4.71**	**0.43**	**ระดับความพึงพอใจสูงสุด**

จากตารางที่ 7.1 คะแนนรวมเฉลี่ยอยู่ที่ **4.71 จาก 5.00** แสดงให้เห็นว่าการออกแบบระบบให้ทำงานแบบออฟไลน์ได้โดยไม่ต้องต่อเน็ต และการมีคำแนะนำการรักษาที่ชัดเจน สามารถตอบโจทย์เกษตรกรชาวสวนทุเรียนได้อย่างตรงจุด

7.4 โค้ดตัวอย่าง: สถาปัตยกรรม Inference Controller ใน Flutter

Listing 7.1 ตัวอย่างโค้ด Dart/Flutter จำลองการทำงานของ Inference Controller เชื่อมต่อ ONNX Runtime

```dart
// durian_inference_controller.dart
//                    Flutter & ONNX Runtime

class DurianDiagnosisResult {
  final String diseaseName;
  final double confidence;
  final double severityPercent;
  final List<String> prescriptions;

  DurianDiagnosisResult({
    required this.diseaseName,
    required this.confidence,
    required this.severityPercent,
    required this.prescriptions,
  });
}

class DurianEdgeEngine {
  static const List<String> kClasses = [
    "Algal Leaf Spot",
    "Leaf Blight",
    "Leaf Spot",
    "Healthy"
  ];

  ///                  (Float32List    1x3x224x224)
  Future<DurianDiagnosisResult> runInference(List<double>
      normalizedPixels) async {
    final stopwatch = Stopwatch()..start();

    // 1.       ONNX Runtime Session
    // final outputs = await
        onnxSession.run([normalizedPixels]);
    //                 Softmax
    final List<double> mockProbabilities = [0.012, 0.984,
        0.003, 0.001];

    int maxIdx = 0;
    double maxProb = mockProbabilities[0];
```

```
    for (int i = 1; i < mockProbabilities.length; i++) {
      if (mockProbabilities[i] > maxProb) {
        maxProb = mockProbabilities[i];
        maxIdx = i;
      }
    }

    stopwatch.stop();
    print("[*] Inference Time:
        ${stopwatch.elapsedMilliseconds} ms");

    // 2.        Prescription Engine
    final prescriptions = _getPrescription(kClasses[maxIdx]);

    return DurianDiagnosisResult(
      diseaseName: kClasses[maxIdx],
      confidence: maxProb * 100.0,
      severityPercent: 14.5,
      prescriptions: prescriptions,
    );
  }

  List<String> _getPrescription(String condition) {
    switch (condition) {
      case "Leaf Blight":
        return [
          "                    ",
          "                          ",
          "               ",
          "        7          "
        ];
      case "Algal Leaf Spot":
        return [
          "                        ",
          "                   "
        ];
      default:
        return ["                  "];
    }
  }
}
```

7.5 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำ ไป ใช้ งาน จริง

ตัวอย่าง การ คำนวณ ที่ 7.1: การ คำนวณ ปริมาณ สาร ชีว ภัณฑ์ ควบคุม โรค เฉพาะ จุด

โจทย์: จาก การ สแกน ด้วย แอปพลิเคชัน ใน แปลง ทุเรียน ขนาด 10 ไร่ (มี ทุเรียน ทั้งหมด 250 ต้น) ตรวจ พบ ต้น ที่ แสดง อาการ โรค ใบ ติด ระดับ ปานกลาง จำนวน 25 ต้น ตาม คำแนะนำ ต้อง ฉีดพ่น สาร ชีว ภัณฑ์ *Bacillus amyloliquefaciens* ใน อัตรา 50 มิลลิลิตร ต่อ น้ำ 20 ลิตร โดย แต่ละ ต้น ที่ มี อาการ ต้อง ใช้ น้ำยา พ่น เฉพาะ ทรง พุ่ม เฉลี่ย 3.5 ลิตร ต่อ ต้น จง คำนวณ ปริมาณ สาร ชีว ภัณฑ์ และ ปริมาณ น้ำ ที่ ต้อง ใช้ เปรียบเทียบ กับ การ พ่น ครอบคลุม ทั้ง 250 ต้น แบบ ดั้งเดิม

1. วิธี การ เฉพาะ จุด ตาม AI (Site-Specific):

$$\text{ปริมาณน้ำรวม} = 25 \text{ ต้น} \times 3.5 \text{ ลิตร} = 87.5 \text{ ลิตร}$$

$$\text{ปริมาณชีวภัณฑ์ที่ใช้} = 87.5 \text{ ลิตร} \times \left(\frac{50 \text{ มล.}}{20 \text{ ลิตร}} \right) = 87.5 \times 2.5 = 218.75 \text{ มิลลิลิตร}$$

2. วิธี การ พ่น เหมา ทั้ง แปลง แบบ เดิม:

$$\text{ปริมาณน้ำเดิม} = 250 \text{ ต้น} \times 3.5 \text{ ลิตร} = 875 \text{ ลิตร}$$

$$\text{ปริมาณชีวภัณฑ์เดิม} = 875 \times 2.5 = 2,187.5 \text{ มิลลิลิตร}$$

สรุป ผล การ ประหยัด: การ ใช้ ระบบ AI ชี้ เป้า ช่วย ลด การ ใช้ สาร ชีว ภัณฑ์ และ ปริมาณ น้ำ ลง ถึง **90%** ประหยัด ต้นทุน ค่า ยา และ แรงงาน ได้ อย่าง มหาศาล

ตัวอย่างการนำไปใช้งานจริงที่ 7.1: ผลกระทบเชิงเศรษฐกิจในวิสาหกิจชุมชนทุเรียนคุณภาพ

วิสาหกิจชุมชนผู้ปลูกทุเรียนแปลงใหญ่ อ.มะขาม จ.จันทบุรี ได้นำแอปพลิเคชันไปทดลองใช้กับสมาชิกจำนวน 30 ครัวเรือน ในรอบฤดูกาลผลิตปี พ.ศ. 2568–2569
ผลสรุปการประเมินรอบ 6 เดือน:

1. **ลดต้นทุนสารเคมีเกษตร:** สมาชิกสามารถลดค่าใช้จ่ายสารเคมีและชีวภัณฑ์เฉลี่ย 4,200 บาทต่อไร่
2. **รักษาคุณภาพใบก่อนออกดอก:** ต้นทุเรียนมีดัชนีพื้นที่ใบสมบูรณ์เฉลี่ยสูงกว่าปีก่อนหน้า 15.2% ทำให้การติดผลสมบูรณ์และลดปัญหาผลแตก/ผลบิดเบี้ยว
3. **การสร้างแผนที่ระบาดวิทยาแปลง (Epidemiological Orchard Map):** ข้อมูล GPS ที่แอปพลิเคชันบันทึกไว้ ช่วยให้ประธานกลุ่มวิสาหกิจสามารถเปิดดูแผนที่ความเสี่ยงบนหน้าจอคอมพิวเตอร์และประกาศเตือนภัยล่วงหน้าแก่สมาชิกในแนวลมได้อย่างแม่นยำ

คำถามทบทวนท้ายบทที่ 7

1. จงอธิบายโครงสร้าง 4 เลเยอร์ของสถาปัตยกรรมแอปพลิเคชัน Mobile Edge AI
2. เหตุใดเฟรมเวิร์ก Flutter จึงเป็นตัวเลือกที่เหมาะสมในการพัฒนาแอปพลิเคชันสำหรับเกษตรกร
3. การออกแบบส่วนต่อประสานผู้ใช้ (UI/UX) สำหรับใช้งานกลางแปลงแดดจ้า ต้องคำนึงถึงปัจจัยใดบ้าง
4. การคำนวณร้อยละความรุนแรงของโรคเชื่อมโยงกับระบบ Prescription Engine อย่างไร
5. จงอธิบายบทบาทของระบบบันทึกพิกัดภูมิศาสตร์ (GPS Geotagging) ในการจัดทำแผนที่โรคระบาด

6. จากตารางผลการประเมิน TAM มิติใดได้รับคะแนนเฉลี่ยสูงสุด และสะท้อนถึงสิ่งใด
7. เพราะเหตุใดการทำงานแบบออฟไลน์ (Offline-First) จึงเป็นหัวใจสำคัญของแอปพลิเคชันเกษตรกรรม
8. จากตัวอย่างการคำนวณที่ 7.1 การฉีดพ่นเฉพาะจุด (Site-Specific Spraying) ส่งผลดีต่อสิ่งแวดล้อมอย่างไร
9. หากกล้องสมาร์ทโฟนของเกษตรกรมีเซนเซอร์ความละเอียดต่ำมาก ระบบควรมีกลไกแจ้งเตือนอย่างไร
10. จงออกแบบแนวทางการเชื่อมต่อข้อมูลจากสมาร์ทโฟนไปยังระบบคลาวด์ส่วนกลางเมื่อโทรศัพท์กลับมาเชื่อมต่อ Wi-Fi

กิจกรรมฝึกปฏิบัติการประจำบทที่ 7

ปฏิบัติการที่ 7.1: การติดตั้งและทดสอบแอปพลิเคชันสมาร์ทโฟนในแปลงจริง

วัตถุประสงค์: เพื่อให้ผู้เรียนสามารถติดตั้งไฟล์แอปพลิเคชัน APK บนมือถือและนำไปทดสอบสแกนใบพืชจริง
ขั้นตอนการปฏิบัติ:

1. ถ่ายโอนไฟล์ `durian_ai_app.apk` ไปยังสมาร์ทโฟนระบบปฏิบัติการ Android
2. ทำการติดตั้งและอนุญาตสิทธิ์การเข้าถึงกล้องและตำแหน่งพิกัด GPS
3. เดินสำรวจต้นทุเรียนในแปลงทดลอง ทำการสแกนใบพืช 5 ใบ และตรวจสอบผลการวินิจฉัยบนหน้าจอ
4. ตรวจสอบหน้าประวัติการบันทึก (History Screen) ว่ามีพิกัดและคำแนะนำการรักษาครบถ้วนหรือไม่
5. สรุปผลการทดสอบและข้อเสนอแนะในการปรับปรุงความเร็วหรือความแม่นยำของระบบ

บทที่ A

คู่มือการติดตั้งและตั้งค่าสภาพแวดล้อมการทำงาน

ภาคผนวกนี้รวบรวมขั้นตอนการติดตั้งและกำหนดค่าสภาพแวดล้อมการ ทำงาน (Development Environment Setup) สำหรับ การ พัฒนา และ ทดสอบ ระบบ ปัญญา ประดิษฐ์ วินิจฉัย โรค ใบ ทุเรียน ทั้ง บน ระบบ คลา วด์ Google Colab และบนเครื่องคอมพิวเตอร์ส่วนบุคคล

A.1 การเตรียมสภาพแวดล้อมบน Google Colab (Cloud GPU)

Google Colab เป็นสภาพแวดล้อมที่เหมาะสมที่สุดสำหรับผู้เริ่มต้นที่ไม่มีการ์ดจอ GPU โดยมีขั้นตอนการตั้งค่าดังนี้:

1. **การสร้างสมุดงานและเลือกการ์ดจอ GPU:**
   - เข้าสู่เว็บไซต์ `https://colab.research.google.com` และลงชื่อเข้าใช้ด้วยบัญชี Google
   - ไปที่เมนู *Runtime* → *Change runtime type*
   - ในช่อง *Hardware accelerator* เลือก **T4 GPU** และกดปุ่ม *Save*
2. **การติดตั้งและอัปเดตแพ็กเกจที่จำเป็น:** พิมพ์คำสั่งต่อไปนี้ในเซลล์แรกของสมุดงาน:

```
!pip install --upgrade torch torchvision torchaudio
!pip install onnx onnxruntime opencv-python matplotlib
    pillow torchinfo
```

3. **การตรวจสอบสถานะการเชื่อมต่อ GPU:**

```
import torch
print(f"[*] PyTorch Version: {torch.__version__}")
print(f"[*] CUDA Available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"[*] GPU Device Name:
        {torch.cuda.get_device_name(0)}")
```

A.2 การติดตั้งสภาพแวดล้อมบนเครื่องคอมพิวเตอร์ส่วนบุคคล (Local Machine)

สำหรับผู้พัฒนาที่ต้องการรันระบบบนเครื่องคอมพิวเตอร์ส่วนบุคคล (รองรับทั้ง Windows, macOS และ Linux):

1. **การสร้างสภาพแวดล้อมเสมือน (Python Virtual Environment):**

```
#
python3 -m venv venv_durian

#            (macOS/Linux)
source venv_durian/bin/activate

#            (Windows PowerShell)
.\venv_durian\Scripts\Activate.ps1
```

2. **การติดตั้งไลบรารีจากไฟล์ requirements.txt:**

```
pip install -r requirements.txt
```

A.3 การ ติด ตั้ง ชุด พัฒนา Flutter สำหรับ การ สร้าง แอปพลิเคชันมือถือ

การสร้างแอปพลิเคชันสมาร์ทโฟนสำหรับติดตั้งบนระบบปฏิบัติการ Android และ iOS มีขั้นตอนดังนี้:

1. ดาวน์โหลด Flutter SDK เวอร์ชันล่าสุดจากเว็บไซต์ทางการ `https://flutter.dev`
2. แตกไฟล์และเพิ่มพาธของโฟลเดอร์ `flutter/bin` เข้าสู่ System Environment Variables (PATH)
3. เปิดเทอร์มินัลและรันคำสั่งตรวจสอบความพร้อม: `flutter doctor`
4. ติดตั้งปลั๊กอินสำหรับ ONNX Runtime ในไฟล์ `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.10.5
  onnxruntime: ^1.2.0
  path_provider: ^2.1.2
```

บทที่ B

ตารางค่าพารามิเตอร์ของแบบจำลองและการปรับแต่ง

ภาคผนวกนี้รวบรวมรายละเอียดพารามิเตอร์ทั้งหมดของการฝึกฝนโมเดล MobileNetV3-Small โครงสร้างเลเยอร์ และการกำหนดค่าในระบบ Explainable AI เพื่อให้สามารถทำซ้ำผลการทดลอง (Reproducibility) ได้อย่างสมบูรณ์

B.1 ตา ราง ค่า ไฮ เปอร์ พารามิเตอร์ สำหรับ การ ฝึกฝน โมเดล (Training Configurations)

ตารางที่ B.1 รายละเอียดการตั้งค่าไฮเปอร์พารามิเตอร์สำหรับการฝึกฝนโมเดล MobileNetV3-Small

ชื่อพารามิเตอร์	ค่าที่กำหนดใช้งาน	คำอธิบายเหตุผลทางเทคนิค
Image Resolution	$224 \times 224 \times 3$	ขนาดมาตรฐานที่สมดุลระหว่างความละเอียดและขนาดเทนเซอร์
Batch Size	32	ขนาดบัตช์ที่เหมาะสมกับหน่วยความจำ GPU และการคำนวณ Gradient
Optimizer	AdamW	ใช้อัลกอริทึม Adam ร่วมกับ Decoupled Weight Decay
Initial Learning Rate	5×10^{-4} (0.0005)	อัตราการเรียนรู้เริ่มต้นที่เหมาะสมสำหรับการทำ Fine-tuning
Learning Rate Scheduler	CosineAnnealingLR	ค่อยๆ ปรับลด LR ตามแนวคลื่นโคไซน์สู่ค่าต่ำสุด 1×10^{-6}
Weight Decay	1×10^{-2} (0.01)	ควบคุมค่าสัมประสิทธิ์ L_2 Regularization ป้องกัน Overfitting
Dropout Probability	0.30	สุ่มตัดการเชื่อมต่อร้อยละ 30 ในชั้น Classifier Head
Loss Function	CrossEntropyLoss	ใช้คำนวณความสูญเสียแบบหลายคลาสร่วมกับ Softmax
Epochs	25	จำนวนรอบการเทรนที่โมเดลลู่เข้าสู่ค่าความแม่นยำสูงสุด
Early Stopping Patience	7 Epochs	หยุดการเทรนทันทีหาก Validation Loss ไม่ดีขึ้นต่อ

B.2 ตารางโครงสร้างเลเยอร์ของโมเดล MobileNetV3-Small

ตารางที่ B.2 โครงสร้างสถาปัตยกรรมภายในและขนาด Feature Map ของ MobileNetV3-Small

บล็อก / เลเยอร์	ขนาด Input	เคอร์เนล / ขยาย	SE Module	ขนาด Output
Conv2d 3x3	$224 \times 224 \times 3$	Stride 2	ไม่ใช้	$112 \times 112 \times 16$
bneck (Inverted)	$112 \times 112 \times 16$	k3 / exp 16	มี (SE)	$56 \times 56 \times 16$
bneck	$56 \times 56 \times 16$	k3 / exp 72	ไม่ใช้	$28 \times 28 \times 24$
bneck	$28 \times 28 \times 24$	k3 / exp 88	ไม่ใช้	$28 \times 28 \times 24$
bneck	$28 \times 28 \times 24$	k5 / exp 96	มี (SE)	$14 \times 14 \times 40$
bneck	$14 \times 14 \times 40$	k5 / exp 240	มี (SE)	$14 \times 14 \times 40$
bneck	$14 \times 14 \times 40$	k5 / exp 120	มี (SE)	$14 \times 14 \times 48$
bneck	$14 \times 14 \times 48$	k5 / exp 144	มี (SE)	$14 \times 14 \times 48$
bneck	$14 \times 14 \times 48$	k5 / exp 288	มี (SE)	$7 \times 7 \times 96$
bneck	$7 \times 7 \times 96$	k5 / exp 576	มี (SE)	$7 \times 7 \times 96$
Conv2d 1x1	$7 \times 7 \times 96$	Stride 1	ไม่ใช้	$7 \times 7 \times 576$
AdaptiveAvgPool	$7 \times 7 \times 576$	Global Pool	-	$1 \times 1 \times 576$
Linear Head 1	$1 \times 1 \times 576$	Hard-	-	1024

บทที่ C

พจนานุกรมคำศัพท์เทคนิคและนิยามศัพท์

ภาคผนวกนี้รวบรวมคำศัพท์เทคนิคเฉพาะทางด้านคอมพิวเตอร์วิทัศน์ การเรียนรู้เชิงลึก พยาธิวิทยาพืช และเกษตรแม่นยำ พร้อมคำนิยามและความหมายภาษาไทย

Backpropagation (การแพร่ย้อนกลับ)

กระบวนการคำนวณอนุพันธ์ย่อยของฟังก์ชันการสูญเสียเทียบกับค่าน้ำหนักทุกตัวในโครงข่ายประสาทเทียม เพื่อใช้ในการปรับปรุงค่าน้ำหนักผ่านขั้นตอนวิธี Gradient Descent

Batch Normalization (การปรับมาตรฐานบัตช์)

เทคนิคการปรับค่าข้อมูลในมินิบัตช์ให้มีค่าเฉลี่ยเป็นศูนย์และส่วนเบี่ยงเบนมาตรฐานเป็นหนึ่ง เพื่อช่วยลดปัญหา Internal Covariate Shift และทำให้โมเดลลู่เข้าเร็วขึ้น

Bounding Box (กรอบสี่เหลี่ยมระบุพิกัด)

พิกัดสี่เหลี่ยม $(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$ ที่ใช้ระบุตำแหน่งและขอบเขตของวัตถุหรือรอยโรคบนภาพดิจิทัล

Confusion Matrix (เมทริกซ์ความสับสน)

ตารางสรุปผลการทำนายของโมเดลเทียบกับค่าความจริง ซึ่งประกอบด้วยช่อง True Positive, False Positive, True Negative และ False Negative

Depthwise Separable Convolution (การแยกมิติคอนโวลูชันเชิงลึก)

เทคนิค การ แยก การ คำนวณ คอน โว ลู ชัน ออก เป็น Depthwise Conv (กรอง เชิง พื้นที่) และ Pointwise Conv (รวมช่องสัญญาณ) เพื่อลดปริมาณการคำนวณลง 8–9 เท่า

Early Stopping (การหยุดก่อนกำหนด)

เทคนิค Regularization ที่ติดตามค่า Validation Loss และยุติการฝึกฝนทันทีเมื่อผลการประเมินเริ่มแย่ลงต่อเนื่องตามจำนวนรอบที่กำหนด เพื่อป้องกัน Overfitting

Edge Computing (การประมวลผลบนขอบ)

สถาปัตยกรรมการประมวลผลข้อมูลที่ดำเนินการบนอุปกรณ์ปลายทางโดยตรง เช่น สมาร์ทโฟน หรือบอร์ดสมองกลฝังตัว โดยไม่ต้องพึ่งพาเซิร์ฟเวอร์คลาวด์

Explainable AI - XAI (ปัญญาประดิษฐ์อธิบายได้)

กลุ่มเทคนิคทางปัญญาประดิษฐ์ที่มุ่งเน้นการเปิดเผยเหตุผล กลไก หรือพื้นที่ที่โมเดลใช้ในการตัดสินใจ เพื่อสร้างความโปร่งใสและความน่าเชื่อถือให้แก่มนุษย์

Focal Loss (ฟังก์ชันการสูญเสียแบบโฟคอล)

ฟังก์ชัน Loss ที่ปรับปรุงจาก Cross-Entropy โดยเพิ่มตัวคูณถ่วงน้ำหนักเพื่อลดความสำคัญของตัวอย่างที่จำแนกได้ง่าย และเพิ่มความสำคัญให้กับตัวอย่างที่จำแนกได้ยาก

Grad-CAM (การสร้างแผนที่ความร้อนถ่วงน้ำหนักเกรเดียนต์)

เทคนิค XAI ที่ใช้เกรเดียนต์ของคลาสเป้าหมายเทียบ

กับเลเยอร์คอนโวลูชันสุดท้าย เพื่อสร้างภาพความร้อนระบุบริเวณที่ส่งผลต่อการตัดสินใจ

Inverted Residuals (โครงสร้างตกค้างแบบกลับด้าน)

บล็อกโครงสร้างใน MobileNetV2/V3 ที่ทำการขยายช่องสัญญาณก่อนทำการคอนโวลูชันเชิงลึก จากนั้นจึงบีบอัดกลับลงมา เพื่อรักษาข้อมูลไม่ให้สูญหาย

Mycelium (เส้นใยรา)

โครงสร้างเส้นสายของเชื้อราที่เจริญแทรกซึมเข้าสู่เนื้อเยื่อพืชเพื่อดูดซับสารอาหารและปล่อยเอนไซม์ย่อยสลายเซลล์

ONNX - Open Neural Network Exchange

มาตรฐานเปิดสำหรับแลกเปลี่ยนและจัดเก็บโมเดลโครงข่ายประสาทเทียม ช่วยให้สามารถถ่ายโอนโมเดลข้ามเฟรมเวิร์กและรันบนเอนจินต่างๆ ได้อย่างอิสระ

Quantization (การลดทอนความแม่นยำตัวเลข)

กระบวนการ แปลง ค่า พารามิเตอร์ ของ โมเดล จาก จำนวนจริง Floating-point (FP32) ไป เป็น จำนวนเต็ม Integer (INT8) เพื่อ ลด ขนาด ไฟล์ และ เพิ่มความเร็วในการประมวลผล

Squeeze-and-Excitation - SE Module

กลไกความใส่ใจที่คำนวณและปรับค่าน้ำหนักความสำคัญ ของ แต่ละ ช่อง สัญญาณ (Channel-wise Attention) โดยอัตโนมัติ

Transfer Learning (การเรียนรู้แบบถ่ายโอน)

กระบวนการ นำ โครง ข่าย ประสาท เทียม ที่ ผ่าน การฝึกฝนบนชุดข้อมูลขนาดใหญ่มาปรับใช้กับงานใหม่ที่มีข้อมูลจำกัด โดยคงค่าน้ำหนักตัวสกัดคุณลักษณะเดิมไว้

บทที่ D

พรอมต์ปัญญาประดิษฐ์เชิงสร้างสรรค์สำหรับสื่อและกิจกรรม

ภาค ผนวก นี้ รวบรวม ชุด คำ สั่ง หรือ พร อมต์ (Prompts) สำหรับ สร้างสรรค์ สื่อ ดิจิทัล และ สื่อ วิดีโอ สั้น ด้วย แบบ จำลอง ปัญญา ประดิษฐ์ เชิง สร้างสรรค์ (Generative Video AI เช่น Google Vids, Google Veo, VideoFX หรือ Runway) ความยาวตอนละ 10 วินาที จำนวน 3 ตอนต่อเนื่อง พร้อมบท บรรยายภาษาไทยสำเนียงจันทบุรีแท้ เพื่อใช้ในการเผยแพร่ผลงานวิจัยและ การประชาสัมพันธ์กิจกรรมอบรมเชิงปฏิบัติการ

D.1 แนวคิด และ หลัก การ ออกแบบ สื่อ วิดีโอ ต่อ เนื่อง (Continuity Workflow)

การสร้างสรรค์วิดีโอสั้นความยาวตอนละ 10 วินาทีให้มีความต่อเนื่อง กลมกลืน (Visual and Narrative Continuity) มีหัวใจสำคัญ 3 ประการ:

1. **การ ควบคุม ทิศทาง ภาพ และ กล้อง (Camera Choreography):** กำหนด ชนิด ของ เลนส์ การ เคลื่อน กล้อง (Pan, Tilt, Crane, Push-in) และอุณหภูมิแสงให้สอดคล้องกันตลอดทั้ง 3 ตอน

2. **การจับคู่อัตลักษณ์ภาพ (Match Cut / Seed Locking):** ล็อกสไตล์ตัวละคร สภาพแวดล้อม และชุดสี เพื่อให้รอยต่อระหว่างตอนที่ 1, 2 และ 3 ลื่นไหลเสมือนเป็นช็อตถ่ายทำในกองเดียวกัน
3. **บทพากย์และเสียงวรรณยุกต์อัตลักษณ์ท้องถิ่น:** การใช้ภาษาไทยถิ่นจันทบุรี และ ระยอง–จันท์–ตราด พร้อม คำ ลงท้าย เอกลักษณ์ เช่น *"ฮิ"*, *"ด็อก"*, *"ก๊ะ"*, *"เนี่ยะ"* ซึ่งสร้างความเข้าถึงง่ายและเป็นมิตรกับชุมชนเกษตรกร

D.2 ชุด พร อมต์ ที่ 1: สื่อ เผย แพร่ นวัตกรรม สวน ทุเรียน อัจฉริยะ AI เมืองจันท์

ชุดวิดีโอความยาวรวม 30 วินาที (10 วินาที $\times$ 3 ตอน) นำเสนอเรื่องราว การนำ เทคโนโลยี ปัญญา ประดิษฐ์ เข้า มาส นับ สนุ นการ ตรวจ วิเคราะห์แปลงทุเรียนภูเขาเมืองจันทบุรี

ตอนที่ 1: ตื่นตาแสงเช้า... ก้าวสู่สวนทุเรียนยุคใหม่ (0–10 วินาที)

วัตถุประสงค์ภาพ: เปิดตัวบรรยากาศสวนทุเรียนเมืองจันท์ยามเช้าและภาพเกษตรกรหนุ่มสาวยกสมาร์ทโฟนเตรียมสแกนแปลง
การเคลื่อนกล้อง: Drone Crane Down & Smooth Push Forward
Prompt ภาษาอังกฤษ (Video Generation):

```
Cinematic ultra-realistic 4K, slow drone establishing
    shot descending over a lush green durian orchard
    on the hills of Chanthaburi in early golden
    morning mist. Sunlight streaming through tree
    canopies. A cheerful Thai local farmer holding a
    smartphone scans fresh durian leaves, gentle
    breeze moving the leaves, highly detailed
    foliage, natural photorealistic lighting, 24fps.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

(น้ำเสียงกระฉับกระเฉง เป็นกันเอง หางเสียงสูงแบบชาวจันท์)
"อรุณสวัสดิ์ยามเช้าที่เมืองจันท์ฮิ! ดูหมอกคลุมยอดเขาสิ

สวยอย่าบอกใครเชียว... สมัยนี้ชาวสวนจันท์เราไม่ได้ดูตาเปล่าแล้วนะเนี่ยะ เปิดมือถือสแกนสวนแต่เช้ากันเลยฮิ!”

ตอนที่ 2: สแกนตรวจโรคแม่นยำ ด้วยปัญญาประดิษฐ์ AI (10–20 วินาที)

วัตถุประสงค์ภาพ: โฟกัสหน้าจอสมาร์ทโฟนที่เล็งไปยังใบทุเรียน ระบบประมวลผลขึ้นกรอบตรวจจับและระดับความเชื่อมั่นแบบ Real-time
การเคลื่อนกล้อง: Smooth Tracking & Macro Pan
Prompt ภาษาอังกฤษ (Video Generation):

```
Continuous match cut from previous scene, macro close-up over-the-shoulder shot of the smartphone screen pointed at a durian branch with mature fruit. Futuristic yet subtle translucent AR bounding boxes and green holographic indicators scanning the leaves and durian texture smoothly, high-tech smart farming interface, organic bokeh background, hyper-detailed, 8k resolution, cinematic color grading.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

(จังหวะตื่นเต้น มั่นใจ ชี้ชวนให้เห็นความแม่นยำของ AI)
“แหม... ดูสิเนี่ยะ! ส่องปุ๊บ กล้อง AI มันรู้ใจปั๊บ โรคราใบติดหรือหนอนเจาะมันตรวจเจอหมดก๊ะ ไม่ต้องมานั่งเดาให้เสียเวลา ทันใจดีแท้ด็อก!”

ตอนที่ 3: รวงทองเมืองจันท์ ทุเรียนเกรดพรีเมียมสู่ครัวโลก (20–30 วินาที)

วัตถุประสงค์ ภาพ: ผลผลิต ทุเรียน หมอน ทอง ผล สวย สมบูรณ์ การเคาะตรวจสอบคุณภาพ และรอยยิ้มภาคภูมิใจของเกษตรกร

การเคลื่อนกล้อง: Medium Shot Arc to Dutch Pan

Prompt ภาษาอังกฤษ (Video Generation):

```
Seamless transition, golden hour light. The farmer
    gently taps a huge, perfect golden-yellow
    Monthong durian hanging from a healthy branch
    with a bamboo stick, crisp sound, showing pride
    with a bright genuine smile. Cut to export crates
    filled with premium durians with GI Chanthaburi
    certification tags, warm and inspiring
    atmosphere, cinematic depth of field, 4K.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

(น้ำ เสียง อบอุ่น ภูมิใจ เชิญ ชวน ลิ้ม ลอง ผลผลิต คุณภาพ)

“นี่แหละฮิ... ทุเรียนพรีเมียมเมืองจันท์ เนื้อเนียน เม็ดลีบ หวานหอมกลมกล่อม ทำด้วยเทคโนโลยีผสมหัวใจชาวสวน ใครได้ชิมเป็นต้องติดใจ... มาเที่ยวเมืองจันท์กันเน้อ!”

D.3 ชุด พร อมต์ ที่ 2: เชิญ ชวน ร่วม กิจกรรม อบรม เชิง ปฏิบัติการพัฒนา AI ทุเรียน

ชุดวิดีโอประชาสัมพันธ์เชิญชวนเข้าร่วมกิจกรรม **“การออกแบบและพัฒนาโมเดลการเรียนรู้เชิงลึก และแอปพลิเคชันสมาร์ทโฟนวิเคราะห์โรคในทุเรียน ด้วยปัญญาประดิษฐ์”** มุ่งเน้นการสร้างแรงบันดาลใจให้แก่ผู้เริ่มต้นและเกษตรกรรุ่นใหม่

ตอนที่ 1: จากปัญหาโรคทุเรียน สู่การเทรน Deep Learning (0–10 วินาที)

วัตถุประสงค์ภาพ: แสดงปัญหาโรคใบติดและไฟทอปธอรา ก่อนตัดสลับเข้าสู่ห้องปฏิบัติการวิจัยคอมพิวเตอร์วิทัศน์ที่กำลังเทรนโมเดล PyTorch
การเคลื่อนกล้อง: Quick Push-in from Diseased Leaf to Monitor Graphs
Prompt ภาษาอังกฤษ (Video Generation):

```
Cinematic dynamic split-to-full shot. Beginning with a close-up of a diseased durian leaf showing leaf blight symptoms, smoothly transitioning into a modern high-tech computer lab. University researchers and smart farmers collaborating on screens showing neural network architectures, PyTorch code, loss curves, and 3D deep learning nodes pulsating with vibrant cyan and green lights. Clean cinematic lighting, 4K resolution, 24fps.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

> **(น้ำเสียงชวนขบคิด จุดประกายความหวังในการแก้ปัญหา)**
> “โรคราใบติด ไฟทอปธอรากวนใจทุเรียนเมืองจันท์อยู่ใช่ไหมฮิ? อย่าเพิ่งท้อเนี่ยะ! มาสร้างโมเดล Deep Learning ด้วยกันกับเรา เทรนให้คอมพิวเตอร์ฉลาดล้ำ ตรวจจับโรคได้แม่นยำแท้ๆ ด็อก!”

ตอนที่ 2: แปลงโมเดลลงมือถือ สแกนตรวจได้ทุกที่ (10–20 วินาที)

วัตถุประสงค์ภาพ: ขั้นตอนแปลงโมเดลสู่สมาร์ทโฟน (Flutter Mobile App) และทดสอบสแกนใบจริงในแปลงโดยไม่ต้องต่ออินเทอร์เน็ต
การเคลื่อนกล้อง: Over-the-shoulder Handheld Tracking Shot
Prompt ภาษาอังกฤษ (Video Generation):

```
Match cut continuity. Close-up hands holding a modern smartphone running a custom mobile app for durian pathology. The camera views a durian tree through the phone screen: real-time AI bounding boxes detect disease classes with high confidence percentages like '98% Healthy' and 'Phytophthora Detected'. Fast-paced coding snippets and Flutter UI designs floating softly in the background. Crisp, sleek, photorealistic, 4K.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

(จังหวะคล่องแคล่ว ตื่นเต้น กระชับ)
“เทรนเสร็จ ก็ยกใส่สมาร์ทโฟนในมือถือเรานี่แหละก๊ะ! ส่องปุ๊บ... ปัญญาประดิษฐ์ตีความปั๊บ รู้ผลทันควันในสวนเลย ไม่ต้องพึ่งอินเทอร์เน็ตแรงๆ ก็สแกนได้สบายปรๆอฮิ!”

ตอนที่ 3: จับมือร่วมเวิร์กชอป ยกระดับทุเรียนจันท์ (20–30 วินาที)

วัตถุประสงค์ภาพ: กลุ่มผู้เข้าร่วมอบรม ทั้งอาจารย์ นักศึกษา และเกษตรกร ยิ้มแย้มและชูสมาร์ทโฟนร่วมกัน พร้อมแบนเนอร์เวิร์กชอป Call-to-Action
การเคลื่อนกล้อง: Crane Pull-back & Slow Arc Up
Prompt ภาษาอังกฤษ (Video Generation):

```
Bright uplifting outdoor-indoor blend shot. A diverse group of young innovators, students, and local farmers smiling together in a lush durian orchard, proudly raising their smartphones displaying the diagnostic app. Soft golden sunlight, inspiring academic workshop banner with text 'AI Durian Smart App Workshop', high energy, warm smiles, commercial grade cinematic color grade, 4K.
```

บทพากย์เสียงภาษาไทย สำเนียงจันทบุรี:

(น้ำเสียงอบอุ่น ทรงพลัง เชื้อเชิญจริงใจ)
“มา เรียน รู้ ด้วย กัน เน้อ... เวิร์ กช อ ปอ อก แบบ Deep Learning และแอปมือถือตรวจโรคทุเรียน! ไม่ยากอย่าง ที่คิด มาร่วมเปลี่ยนสวนเมืองจันท์ให้เป็นสมาร์ทฟาร์ม ระดับโลกด้วยกันฮิ... รีบสมัครกันเข้ามาเลยเน้อ!”

D.4 ข้อแนะนำในการสังเคราะห์เสียงพากย์สำเนียงท้องถิ่น (AI Voiceover Guidelines)

เพื่อให้เสียงพากย์ที่ได้จากระบบ Text-to-Speech (TTS) หรือ Voice Cloning มีชีวิตชีวาและสะท้อนสำเนียงเมืองจันท์อย่างถูกต้อง แนะนำปฏิบัติตามแนวทางต่อไปนี้:

- **การบันทึกต้นแบบเสียง (Voice Cloning Training):** บันทึกไฟล์เสียงตัวอย่างของเจ้าของสำเนียงท้องถิ่นในบรรยากาศห้องเงียบ ความยาวอย่างน้อย 1–2 นาที โดยให้มีตัวอย่างการออกเสียงหางเสียงคำว่า *“ฮิ”*, *“เนี่ยะ”*, *“ก๊ะ”*, *“ด็อก”*, *“เชียว”*
- **การ กำกับ อารมณ์ และ เครื่องหมาย วรรค ตอน:** การ ใช้ เครื่องหมาย อัศเจรีย์ (!) และ จุด ไข่ปลา (...) ใน สคริปต์ ช่วย ปรับ ให้ โมเดล AI ยก จังหวะ เสียง ท้าย ประโยค ให้ สูง ขึ้น ตาม ธรรมชาติ ของ สำเนียง ระยอง–จันท์–ตราด
- **การมิกซ์เสียงประกอบ (Sound Design):** เสริมด้วยเสียงบรรยากาศสวนธรรมชาติ (Ambience of Orchard Birds, Gentle Wind) และเสียงเคาะลูกทุเรียนเบาๆ ในตอนที่ 3 เพื่อเพิ่มความสมจริงระดับมืออาชีพ

บรรณานุกรม

[ภาษาไทย / Thai References]

กรมวิชาการเกษตร. (2565). *คู่มือการวินิจฉัยและป้องกันกำจัดโรคและแมลงศัตรูทุเรียนที่สำคัญในเขตภาคตะวันออก*. กรุงเทพฯ: สำนักวิจัยและพัฒนาการอารักขาพืช กระทรวงเกษตรและสหกรณ์.

จันทมาลี, จ., และ ทัศนา, ช. (2567). การศึกษาสัณฐานวิทยาและการแพร่กระจายของเชื้อสาหร่าย *Cephaleuros virescens* บนใบทุเรียนพันธุ์หมอน ทองใน จังหวัด จันทบุรี. *วารสาร วิทยาศาสตร์ และ นวัตกรรมการเกษตร*, 8(2), 145–158.

ทัศนา, ช., และ จันทมาลี, จ. (2568). *สถาปัตยกรรมไอโอทีและการเรียนรู้เชิงลึกเพื่อการเกษตรแม่นยำในแปลงสวนทุเรียนอัจฉริยะ*. จันทบุรี: คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี.

ทัศนา, ช., สุขสมบูรณ์, ภ., และ วงศ์สุวรรณ, ว. (2568). การพัฒนาระบบประมวลผลบนอุปกรณ์ขอบด้วยเทคนิค Explainable AI สำหรับตรวจจับโรคพืช ผลไม้ เมืองร้อน. *วารสาร วิจัย และ พัฒนา มหาวิทยาลัยราชภัฏรำไพพรรณี*, 19(1), 88–102.

สำนักงานเศรษฐกิจการเกษตร. (2567). *รายงานสถิติการค้าสินค้าเกษตรไทยกับต่างประเทศ ประจำปี 2566*. กรุงเทพฯ: ศูนย์สารสนเทศการเกษตร กระทรวงเกษตรและสหกรณ์.

[ภาษาอังกฤษ / International English References (APA 7th Edition)]

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 1314–1324). https://doi.org/10.1109/ICCV.2019.00140

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*. https:// doi.org/ 10.48550/ arXiv.1704.04861

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 7132–7141). https://doi.org/ 10.1109/CVPR.2018.00745

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 2980–2988). https://doi.org/10.1109/ICCV.2017.324

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1711.05101

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, Article 1419. https:// doi.org/ 10.3389/ fpls.2016.01419

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 618–626). https://doi.org/10.1109/ICCV.2017.74

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)* (pp. 6105–6114). PMLR. https://proceedings.mlr.press/v97/tan19a.html

Thassana, C., & Janthamalee, J. (2025). Explainable artificial intelligence with MobileNetV3 architecture for real-time plant disease detection on smart edge devices. *Computers and Electronics in Agriculture*, 225, Article 109280. https:// doi.org/10.1016/j.compag.2025.109280

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*. https:// doi.org/10.48550/arXiv.1710.09412

ดัชนีสืบค้นคำสำคัญ

AdamW Optimizer, 42

Cephaleuros virescens, 6
Classifier Head, 33
Colletotrichum gloeosporioides, 6
Confusion Matrix, 39, 43
Convolutional Neural Networks, 27
Cross-Entropy Loss, 39, 42

Data Augmentation, 15
DataLoader, 21
Depthwise Separable Convolution, 30

Edge AI, 1, 67
Explainable AI, 53, 56

Flutter Framework, 79
Flutter SDK, 93

Generative AI, 103
Geometric Augmentation, 19
Google Colab, 91
Google Flow, 103
Grad-CAM, 53

HUD Scanner, 83
Hyperparameters, 95

InferenceController, 85
Inverted Residuals, 30

Kernel, 30

Learning Curves, 42

MobileNetV3, 27

ONNX Export, 67
ONNX Runtime, 70
Overfitting, 42

Photometric Augmentation, 19
Precision & Recall, 43
PyTorch Dataset, 21
PyTorch Training, 39
PyTorch Training Loop, 46

Quantization, 67
Quantization INT8, 71

Rhizoctonia solani, 6

Squeeze-and-Excitation, 31

Terminal Execution, 45
TorchScript, 70
Transfer Learning, 27, 33

กล้องสดเรียลไทม์, 82
การกระจายตัวของข้อมูล, 18
การติดตั้งสภาพแวดล้อม, 91
การประมวลผลภาพดิจิทัล, 15
การยอมรับของเกษตรกร, 84
การยอมรับเทคโนโลยี, 79
การสร้างสื่อวิดีโอ, 103
การเปรียบเทียบโมเดล, 32
ข้อจำกัดฮาร์ดแวร์, 70
ความรุนแรงของโรค, 53
คอนโวลูชัน, 30
ชุดข้อมูลภาพใบทุเรียน, 15
ทุเรียนหมอนทอง, 4
นิยามศัพท์, 99
ปฏิทินการระบาด, 8
ปริภูมิสี HSV, 18
ปริภูมิสี RGB, 18
พจนานุกรมคำศัพท์, 99
พยาธิวิทยาพืช, 4
พรอมต์ปัญญาประดิษฐ์, 103
พารามิเตอร์แบบจำลอง, 95
ภาพความร้อน Grad-CAM, 59
ระดับความรุนแรงของโรค, 60
ระบบโครงข่ายประสาท, 7
ระบาดวิทยาตามฤดูกาล, 8
สคริปต์ Python, 9
สถาปัตยกรรม Edge AI, 7
สถาปัตยกรรม Flutter, 82
สภาพแวดล้อมแปลงเปิด, 9
สมการ Grad-CAM, 57
ส่วนต่อประสานผู้ใช้, 83
อุปกรณ์ขอบ, 70
เกณฑ์มาตรฐานฮาร์ดแวร์, 72
เกษตรอัจฉริยะ, 1
แบบจำลอง TAM, 84
แอปพลิเคชันสมาร์ทโฟน, 79
โค้ด Flutter, 85
โค้ด GradCAM, 61
โค้ดส่งออก ONNX, 73
โมเดลกล่องดำ, 56
โรคใบทุเรียน, 1

ประวัติผู้เขียน



ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

ตำแหน่ง ทาง วิชาการ: ผู้ ช่วย ศาสตราจารย์ สาขา วิชา ฟิสิกส์

คณะ วิทยาศาสตร์ และ เทคโนโลยี มหาวิทยาลัย ราชภัฏ รำไพ พรรณี

อีเมล: chewathassana@rbru.ac.th

ประวัติการศึกษา

- **ปร.ด. (ฟิสิกส์ประยุกต์)**
  สถาบันเทคโนโลยีพระจอมเกล้า เจ้าคุณทหารลาดกระบัง
- **วท.ม. (ฟิสิกส์ประยุกต์)**
  สถาบันเทคโนโลยีพระจอมเกล้า เจ้าคุณทหารลาดกระบัง
- **วท.บ. ฟิสิกส์ (คอมพิวเตอร์ และ อิเล็กทรอนิกส์)**
  มหาวิทยาลัยนเรศวร

ความเชี่ยวชาญและงานวิจัย

- ฟิสิกส์เกษตรดิจิทัลและไอโอที (AIoT SmartFarm)
- ปัญญาประดิษฐ์และการเรียนรู้เชิงลึก (Computer Vision)
- การคำนวณโครงสร้างสารแม่เหล็กและควอนตัมฟิสิกส์